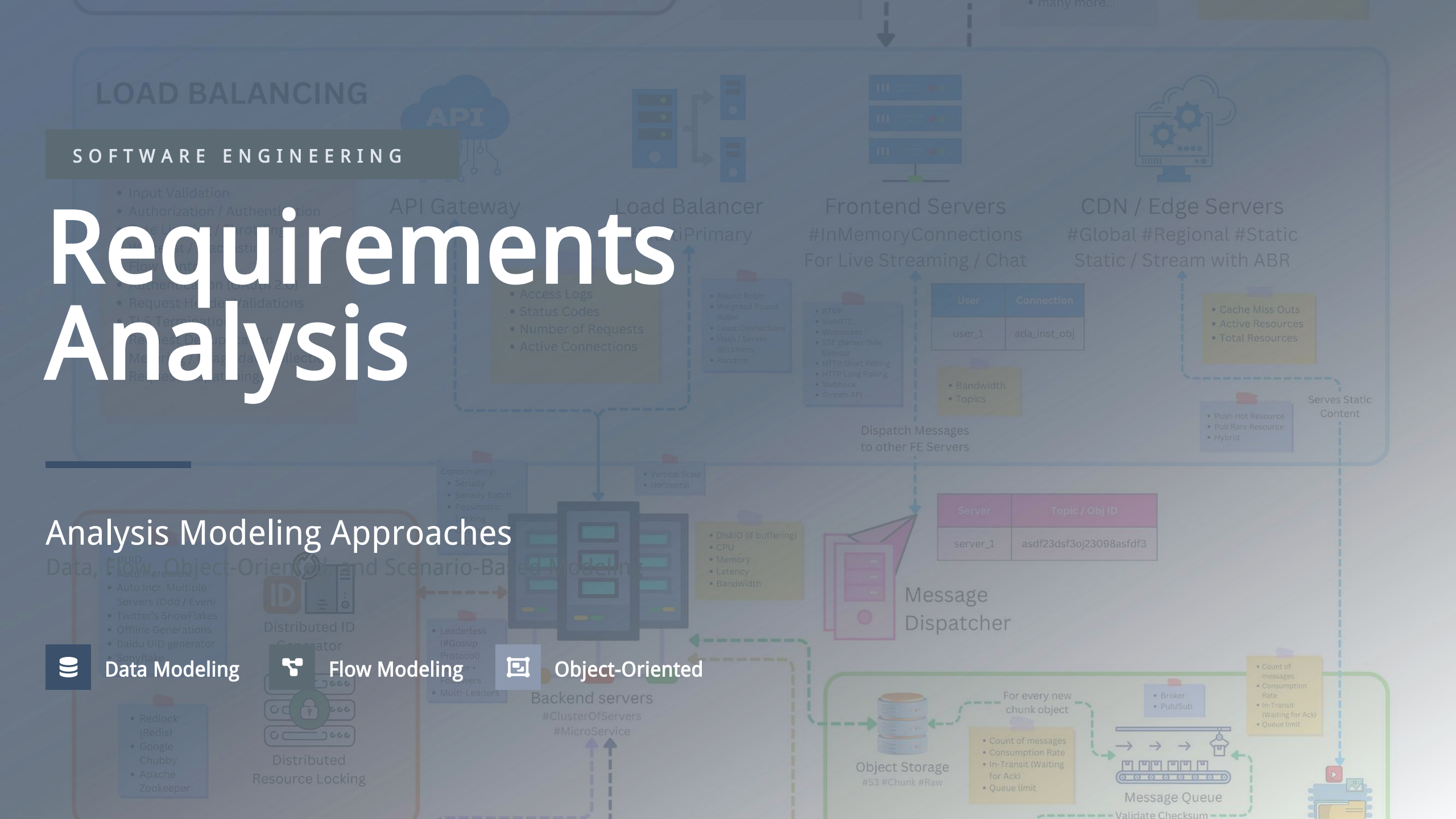
SOFTWARE ENGINEERING

Requirements Analysis

Analysis Modeling Approaches

Data, Flow, Object-Oriented and Scenario-Based Modeling

Data Modeling | Flow Modeling | Object-Oriented



Requirements Analysis

Operational Characteristics

Specifies software's operational characteristics - what the system does and how it behaves.

System Interface

Indicates software's interface with other system elements - how it connects and communicates.

Constraints

Establishes constraints that software must meet - limitations and requirements.

Key Insight: Requirements analysis allows the software engineer (analyst or modeler) to elaborate on basic requirements and build comprehensive models.

The Analyst Role

The software engineer, called an **analyst** or **modeler** in this role, performs critical functions:

1. Elaborate on basic requirements established during earlier requirement engineering tasks
2. Build models that depict user scenarios and functional activities
3. Model problem classes and their relationships
4. Represent system and class behavior
5. Show the flow of data as it is transformed

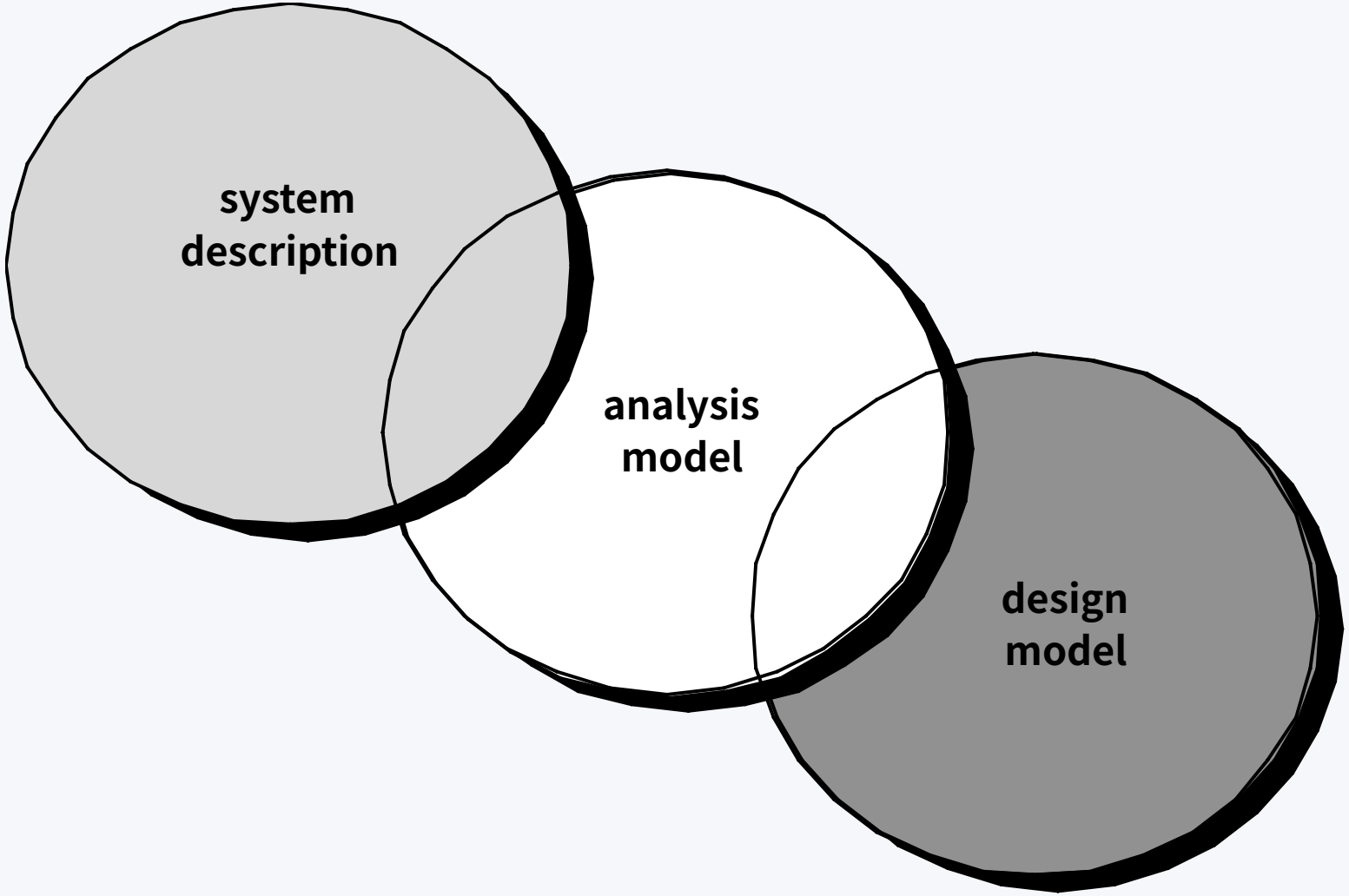
User Scenarios

Class Behavior

Data Flow

A Bridge

Connecting Problem Domain to Solution Space



Rules of Thumb

Best Practices for Analysis Modeling

1

Focus on Domain Requirements

The model should focus on requirements visible within the problem or business domain . The level of abstraction should be relatively high, avoiding implementation details.

2

Add Value with Each Element

Each element of the analysis model should add to overall understanding of software requirements. Every component must provide insight into the information domain, function, and behavior.

3

Delay Infrastructure Considerations

Delay consideration of infrastructure and other non-functional models until design phase. Focus on "what" the system should do, not "how".

4

Minimize Coupling

Minimize coupling throughout the system. Reduce dependencies between model elements to improve maintainability and flexibility.

5

Provide Stakeholder Value

Be certain that the analysis model provides value to all stakeholders . Different audiences should be able to understand and use the model.

6

Keep It Simple

Keep the model as simple as it can be . Avoid unnecessary complexity. The simplest model that captures requirements is the best.

> "Everything should be made as simple as possible, but not simpler." - Albert Einstein

Domain Analysis

Domain analysis is a systematic process of identifying, collecting, organizing, and representing the relevant information in a domain to support software development.

Define

Define the domain to be investigated. Establish boundaries and scope of analysis.

Collect

Collect a representative sample of applications in the domain.

Analyze

Analyze each application in the sample to identify common patterns.

Develop

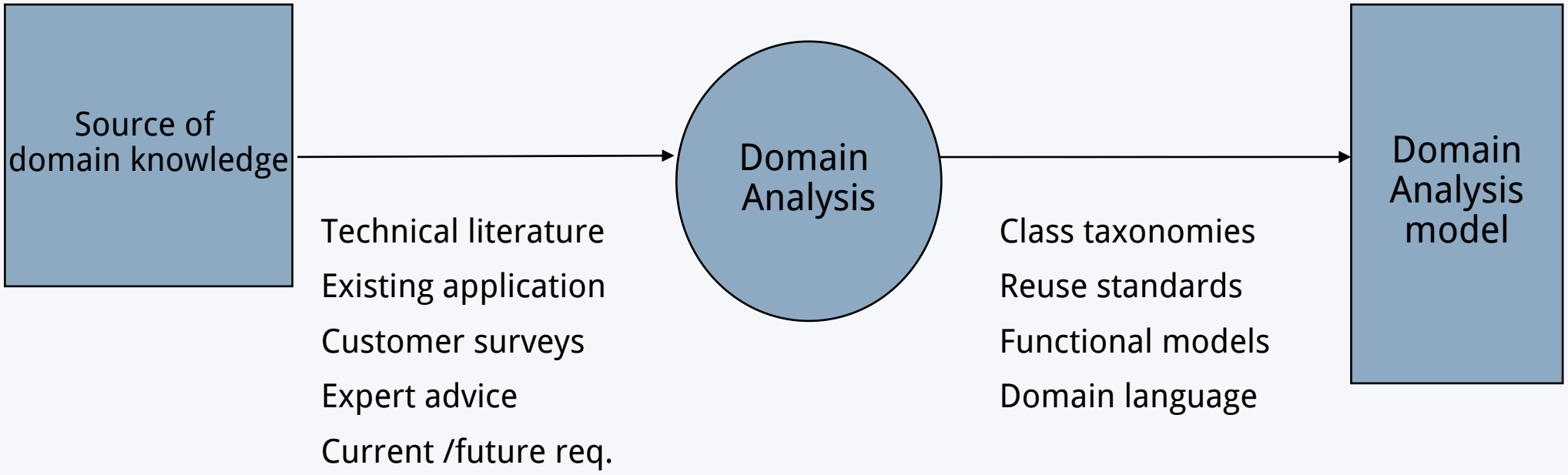
Develop an analysis model for the objects.

Key Benefits of Domain Analysis

- **Reusability**
  Identifies reusable components and patterns
- **Consistency**
  Ensures common vocabulary and models
- **Efficiency**
  Accelerates development through domain understanding

Domain Analysis

Input Sources and Output Models



Analysis Modeling Approaches

Structured and Object-Oriented Methods

Structured Analysis

Focus on **Data and Processes**

- Functional decomposition
- Data flow modeling
- Process specifications

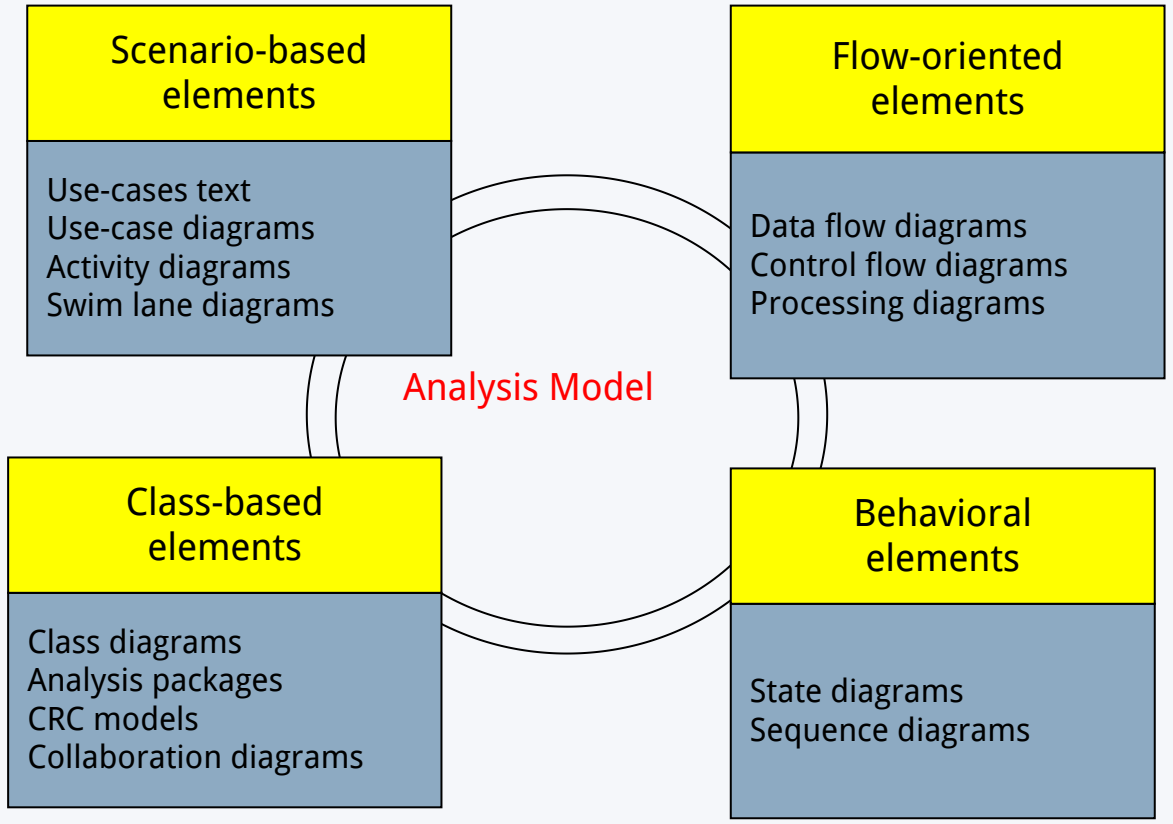
Object-Oriented Analysis

Focus on **Classes and Unified Process (UML)**

- Encapsulation of data and behavior
- Inheritance and polymorphism
- Use-case driven approach

Combined Analysis Model

Modern software engineering combines features of both approaches to leverage strengths of each methodology.



Data Modeling

Examining Data Objects and Their Relationships

What is Data Modeling?

Data modeling is the process of **identifying, defining, and organizing** data objects and their relationships within the system.

Independent of Processing

Examines data objects independently of processing - focuses on "what" data exists.

Customer's Level

Creates a model at the customer's level of abstraction - business concepts.

Focus on Data Domain

Data modeling focuses attention on the data domain , revealing structures, relationships, and constraints.

Relationship Mapping

Indicates how data objects relate to one another through cardinality and modality.

Primary Data Modeling Techniques

ERD
Entity-Relationship Diagrams

Key Principle: Data models should be technology-agnostic, focusing on business concepts rather than implementation details.

What is a Data Object?

Definition and Characteristics

Data Object

A Fundamental Concept in Data Modeling

A data object is something that is described by a **set of attributes** (data items) and that will be **manipulated within the software** (system).

Described by Attributes

Each data object has attributes that describe its characteristics (e.g., a book has ISBN, title, author).

Manipulated by System

The object is used or modified by the software system during operation.

Key Characteristics

Unique Identification

Each instance can be identified uniquely (e.g., ISBN # for books, SSN for persons).

Necessary Role

Each plays a necessary role ; the system couldn't function without access to instances.

Attribute Descriptor

Each is described by attributes that are themselves data items.

Example: Book Object

A book object has attributes: ISBN (unique ID), title, author, publisher, publication date.

Typical Data Objects

Categories of Objects Found in Systems

External Entities

Objects outside the system that interact with it.

Examples: printer, user, sensor

Things

Physical or information objects in the system.

Examples: reports, displays, signals

Occurrences/Events

incidents or actions that occur at a point in time.

Examples: interrupt, alarm, transaction

Roles

Positions or responsibilities played by people.

Examples: manager, engineer, salesperson

Organizational Units

Groups or divisions within an organization.

Examples: division, team, department

Places

Physical locations or areas relevant to the system.

Examples: manufacturing floor, warehouse

Additional Category: Structures

Structures define a class of objects or related classes (e.g., sensors, four-wheeled vehicles, computers, employee records).

Data Objects and Attributes

Understanding the Relationship

A data object contains a **set of attributes** that act as an aspect, quality, characteristic, or descriptor of the object.

- **Object: Entity**
- **Attributes: Properties**
- **Values: Data**

Example: Automobile Object

object: automobile

attributes:
- Make
- Model
- ID number
- Body type
- Color
- Owner

What is a Relationship?

Connecting Data Objects

Relationship

A relationship indicates **"connectedness"** - a "fact" that must be "remembered" by the system.

Cannot Be Computed

A fact that cannot or is not computed mechanically.

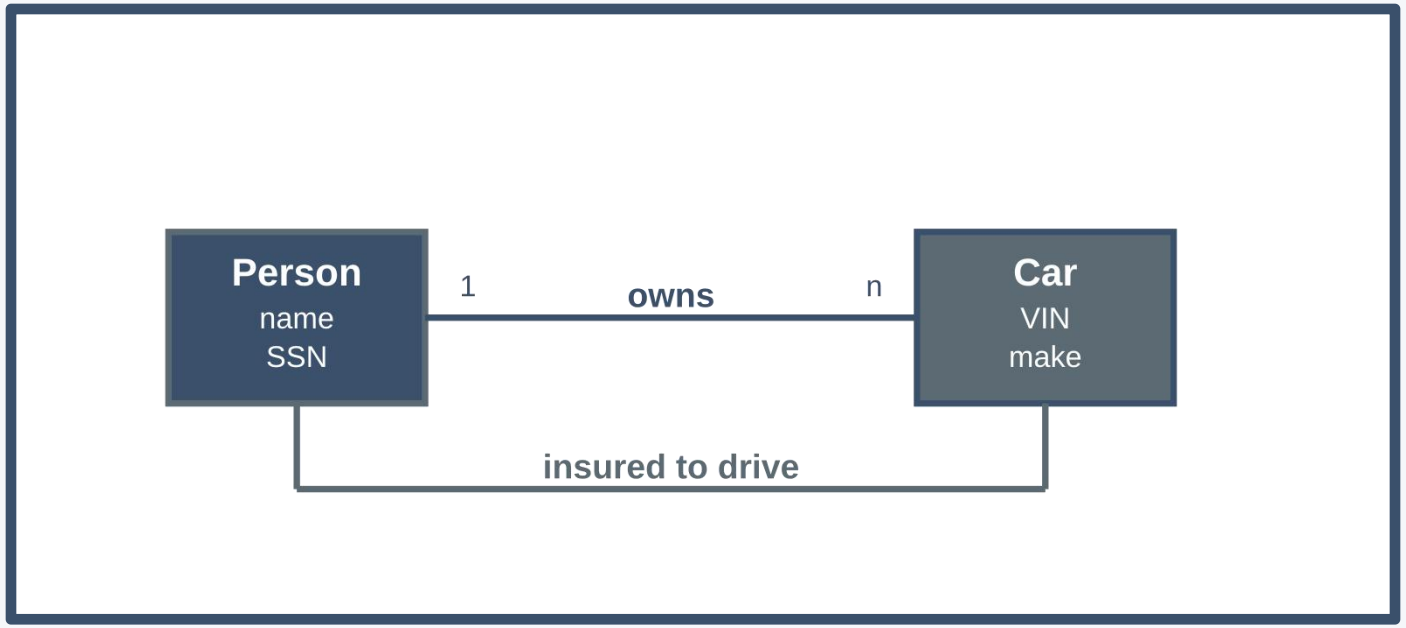
Multiple Instances

Several instances of a relationship can exist.

Multiple Ways

Data objects can be related in many different ways .

Example: Person and Car Relationships



Relationship 1: owns

A Person **owns** a Car. This is a **one-to-many** relationship (one person can own multiple cars).

Relationship 2: insured to drive

A Person is **insured to drive** a Car. This relationship must be "remembered" by the system.

ERD Notation

Entity-Relationship Diagram Cardinality

One-to-One

1:1 Relationship



One instance of Object A relates to exactly one instance of Object B.

Example: Person has one Passport

One-to-Many

1:n Relationship



One instance of Object A relates to multiple instances of Object B.

Example: Customer places many Orders

Many-to-Many

m:n Relationship



Multiple instances of Object A relate to multiple instances of Object B.

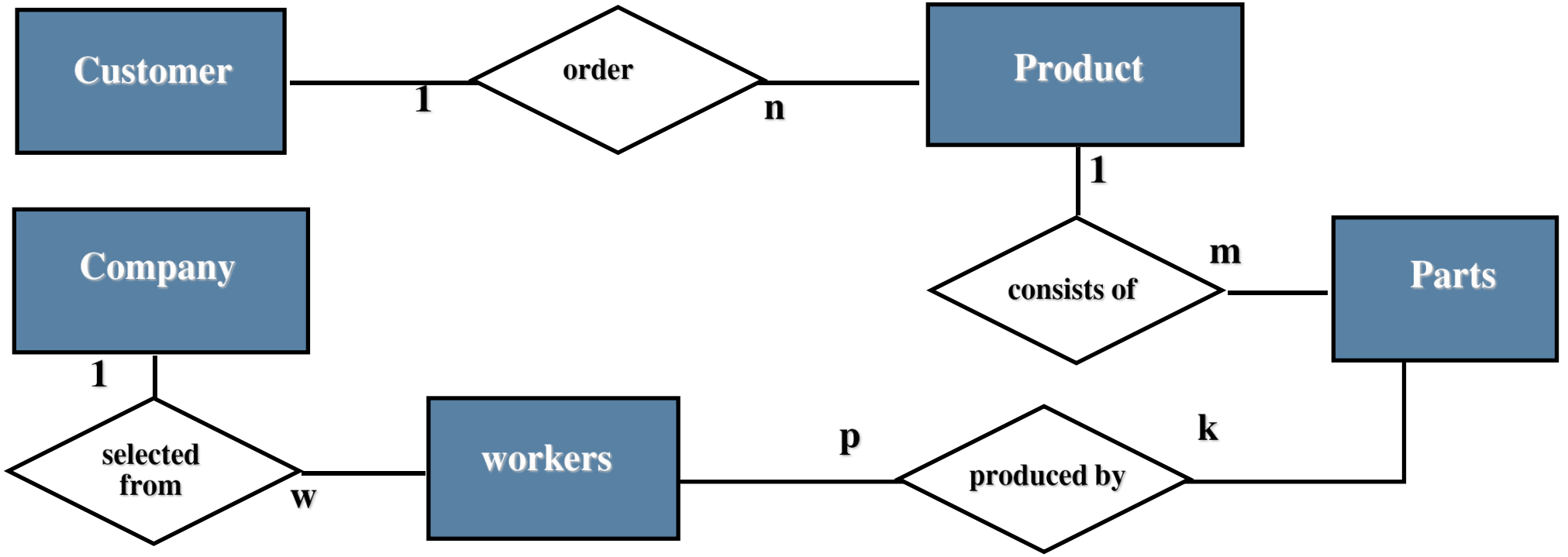
Example: Students enroll in many Courses

Cardinality Notation

Cardinality defines the **number of instances** of one object that relate to instances of another object.

The ERD: An Example

Complex Relationships in a Business System



Flow-Oriented Modeling

Data Flow Diagrams and Process Modeling

Flow-Oriented Modeling

Represents how **data objects are transformed** as they move through the system.

Key Characteristics

- Shows data movement and transformation
- Focuses on processes that change data
- Represents system as information transformer
- Hierarchical decomposition possible

Unique View

Flow-oriented modeling continues to provide a view of the system that is unique - it shows how data flows and transforms.
It should be used to supplement other analysis model elements like use cases and class diagrams.

Data Flow Diagram (DFD)
A diagrammatic form that shows data transformation processes.

"Old School" Approach
Considered by many to be an 'old school' approach but still valuable.

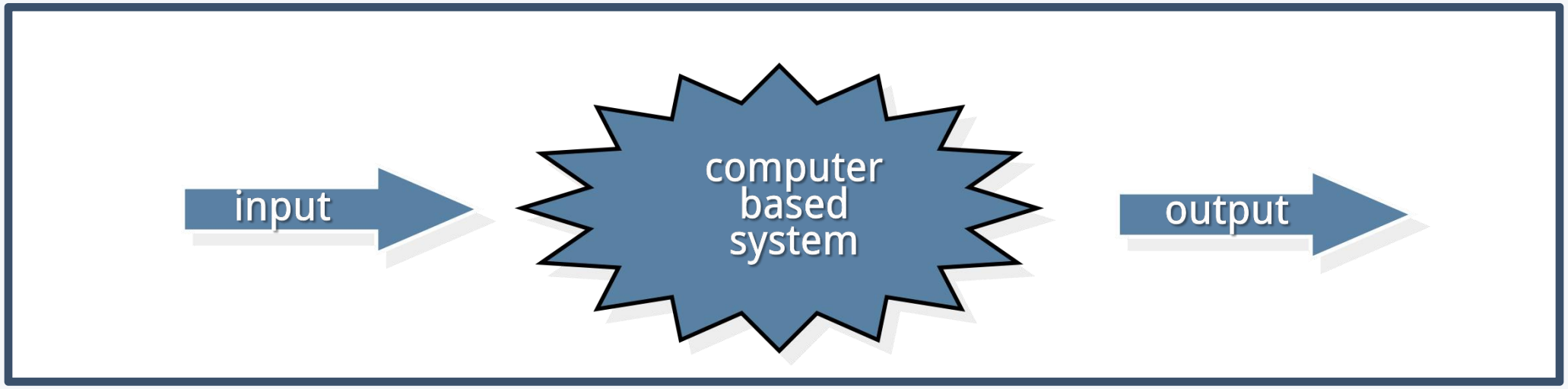
DFD Components

- External Entities
- Processes
- Data Flows
- Data Stores

The Flow Model

System as Information Transformer

> Every computer-based system is an **information transform**...



Input

Raw data enters the system from external sources.

Transformation

System processes transform input data through multiple stages.

Output

Transformed data is delivered to external destinations.

Flow Modeling Notation

DFD Symbols and Their Meanings



External Entity

Producer or Consumer of Data

A source or destination of data that is outside the system boundary .

Examples: person, device, sensor, another system



Process

Data Transformer

A data transformer that changes input to output.

Examples: compute taxes, format report, display graph

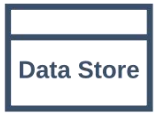


Data Flow

Movement of Data

Data flows through a system, beginning as input and being transformed into output.

Must always be labeled with meaningful names



Data Store

Storage for Later Use

Data is often stored for later use in the system.

Examples: database, file, memory buffer

Key Principle: DFDs show **what** processes data, not **how** - they represent logical flow without implementation details.

External Entity

Sources and Destinations of Data



External Entity

Outside the System

A **producer or consumer of data** that is external to the system being modeled.

Data Must Originate Somewhere

Data must always originate somewhere - it doesn't appear from nowhere.

Data Must Be Sent to Something

Data output must always be sent to something - it doesn't disappear into void.

Examples of External Entities

Person

A user who provides input or receives output from the system.

Device

Hardware devices like printers, scanners, or sensors that interact with the system.

Sensor

Input devices that provide real-time data from the physical environment.

Another System

External computer-based systems that exchange data with the system being modeled.

Important: External entities represent roles, not specific individuals or instances. "Customer" is an entity, but "John Smith" is an instance.

Process

Data Transformer



Process

Data Transformer

A process is a **data transformer** that changes input to output through some form of processing.

Input → Output

Takes input data and transforms it into output data .

Must Always Be Processed

Data must always be processed in some way to achieve system function.

Examples of Processes

Compute Taxes

Takes income, deductions, and tax rates as input; calculates tax amount as output.

Determine Area

Takes dimensions (length, width) as input; calculates and outputs area value.

Format Report

Takes raw data as input; formats, organizes, and outputs structured report.

Display Graph

Takes numerical data as input; generates and displays visual graph as output.

Process Characteristics

Processes should be named with **verb phrases** describing the action (e.g., "Validate Order", "Generate Invoice").

Data Flow

Movement of Data Through the System

Data Flow

Data flows through a system, beginning as input and being transformed into output.

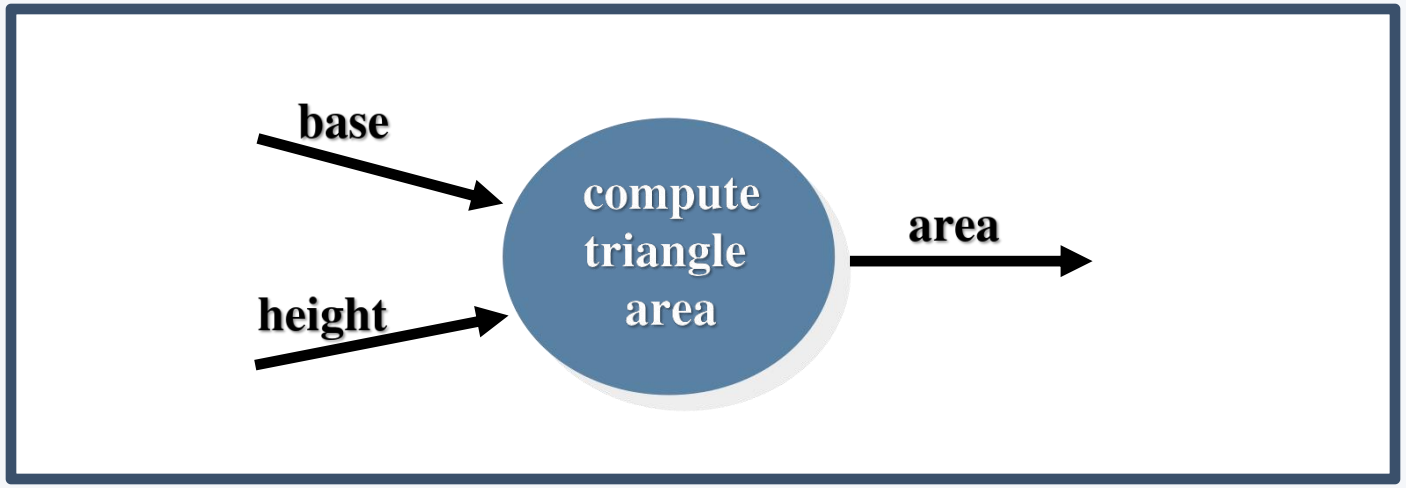
Labeled Arrows

Represented by labeled arrows showing data movement.

Always Labeled

Must be labeled with meaningful names describing the data.

Example: Compute Triangle Area



Input Flows

- base (triangle base length)
- height (triangle height)

Output Flow

- area (computed triangle area)
- Formula: area = 0.5 × base × height

Data Stores

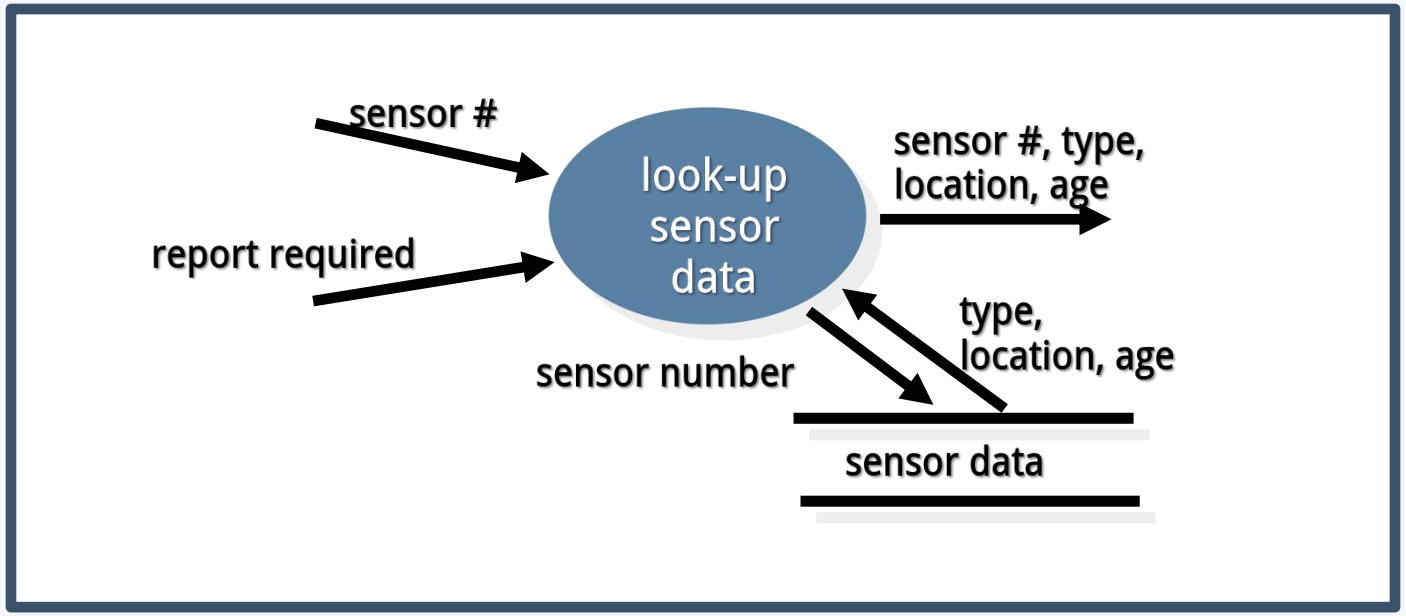
Persistent Data Storage



Data Stores

Data is often **stored for later use** in the system.

Example: Sensor Data Lookup



Persistent Storage

Persistent storage that survives process execution.

Read and Write

Can be read from and written to by processes.

Inputs

- report required (trigger)
- sensor number (key)
- type, location, age (criteria)

Data Store Usage

- Read: sensor data lookup
- Query: find by sensor id
- Output: retrieved sensor data

DFD Guidelines

Best Practices for Data Flow Diagrams

1

Label All Icons

All icons must be labeled with meaningful names that clearly describe the data or process.

2

Evolve Through Levels

The DFD evolves through multiple levels of detail from high-level context to detailed processes.

3

Begin with Context Level

Always begin with a context level diagram (Level 0) that shows the entire system as a single process.

4

Show External Entities at Level 0

Always show external entities at level 0 to define system boundaries.

5

Label Data Flow Arrows

Always label data flow arrows to indicate what data is being transferred.

6

No Procedural Logic

Do not represent procedural logic like loops, decisions, or sequencing in DFDs.

Key Principle

DFDs focus on **data movement and transformation**, not control flow or timing. Show what happens to data, not when or how.

Constructing a DFD — I

Creating a Data Flow Diagram involves a systematic approach to identify and organize system components.

1 Review Data Model

Review the **data model** to isolate data objects and use a grammatical parse to determine "operations".

Key Actions:
- Identify data entities
- Parse requirements text
- Determine operations

2 Determine External Entities

Determine **external entities** - producers and consumers of data.

Key Questions:
- Who provides input?
- Who receives output?
- What systems interact?

3 Create Level 0 DFD

Create a **Level 0 DFD** - the context diagram showing the entire system as one process.

Components:
- Single process (system)
- External entities
- Major data flows

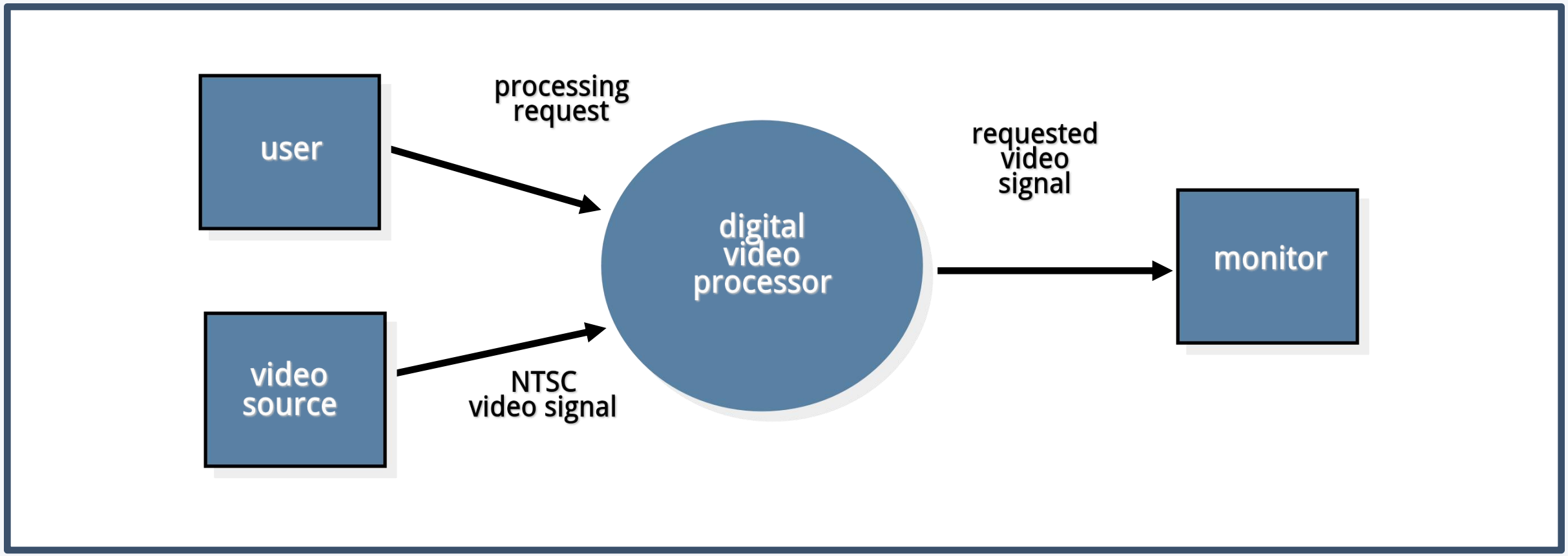


Key Insight

Start **simple** and add detail progressively. The Level 0 DFD provides the big picture before diving into details.

Level 0 DFD Example

Digital Video Processing System



External Entities

- user (requests processing)
- video source (provides input)
- monitor (displays output)

Process

- digital video processor
- Transforms NTSC signal
- Processes based on request

Data Flows

- processing request
- NTSC video signal
- requested video signal

Constructing a DFD—II

Refinement and Decomposition

1 **Write Narrative**

Write a narrative describing the transform - what the process does in business terms.

2 **Parse for Next Level**

Parse to determine next level transforms - identify sub-processes within the main process.

3 **Balance the Flow**

"Balance" the flow to maintain data flow continuity between levels.

4 **Develop Level 1 DFD**

Develop a Level 1 DFD that decomposes the Level 0 process into sub-processes.

5 **Use Expansion Ratio**

Use approximately 1:5 expansion ratio - decompose one process into about 5 sub-processes.

Balancing Rule

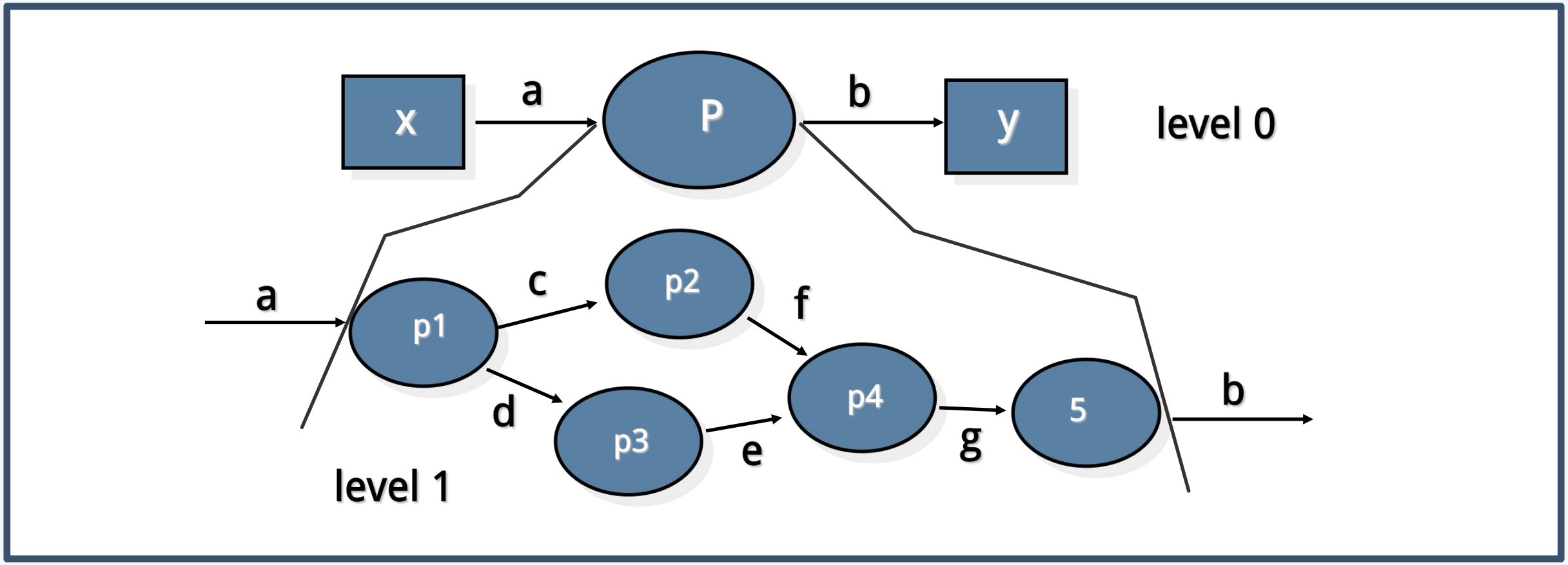
Inputs and outputs at one level must match inputs and outputs at the next level. What goes in and comes out must be preserved across levels.

Decomposition Strategy

Each bubble is refined until it does **just one thing**. Continue decomposing until each process represents a single, well-defined function.

The Data Flow Hierarchy

Decomposition from Level 0 to Level 1



Level 0 (Context)
Single process **P** with inputs **a, b** and outputs **x, y**. Shows system boundary and external interactions.

Level 1 (Decomposed)
Process **P** decomposed into **p1, p2, p3, p4** with internal data flows **d, e, f, g**.

Flow Modeling Notes

Important Principles for DFD Development

1

Refine Until Single Function

Each bubble is refined until it does just one thing - represents a single, coherent function.

2

Expansion Ratio Decreases

The expansion ratio decreases as the number of levels increases - higher levels have fewer sub-processes.

3

Typical Depth: 3 to 7 Levels

Most systems require between 3 and 7 levels for an adequate flow model.

4

Data Flow Expansion

A single data flow item may be expanded as levels increase.

Level 0
Context diagram - entire system as one process

Levels 1-3

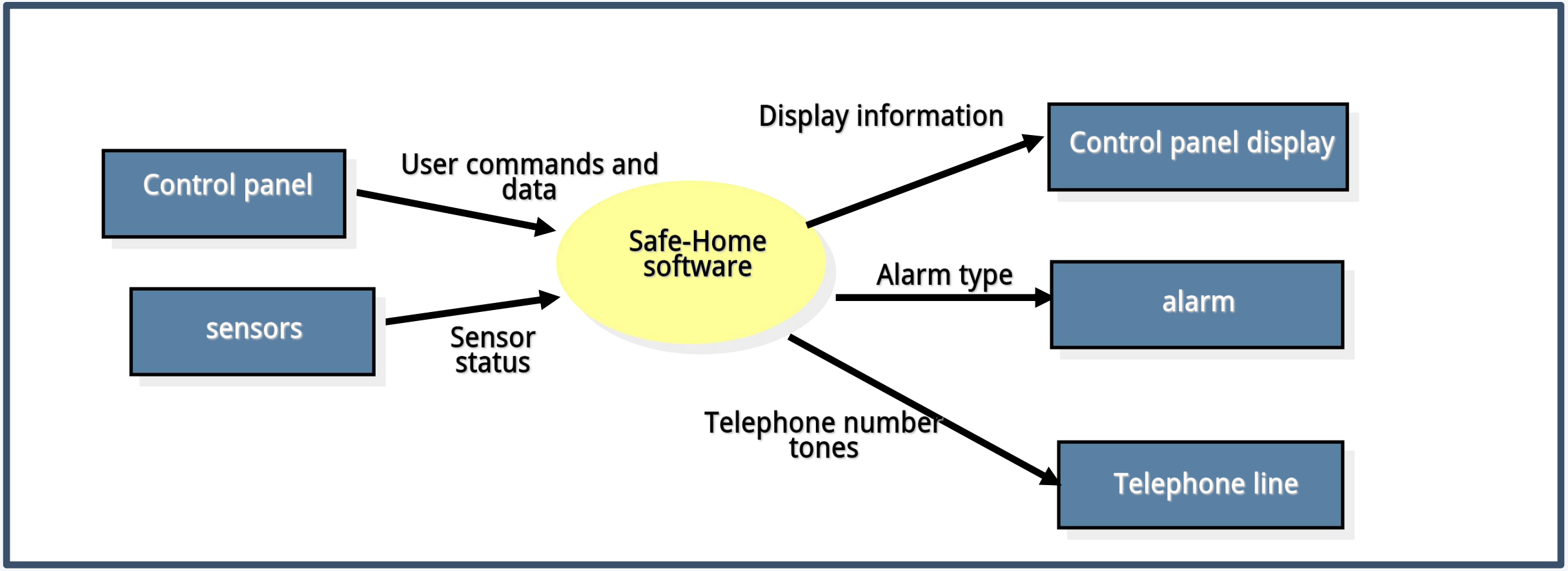
Major subsystems and processes

Levels 4-7

Detailed processes and functions

Level 0 DFD Example

Safe-Home Security System Context

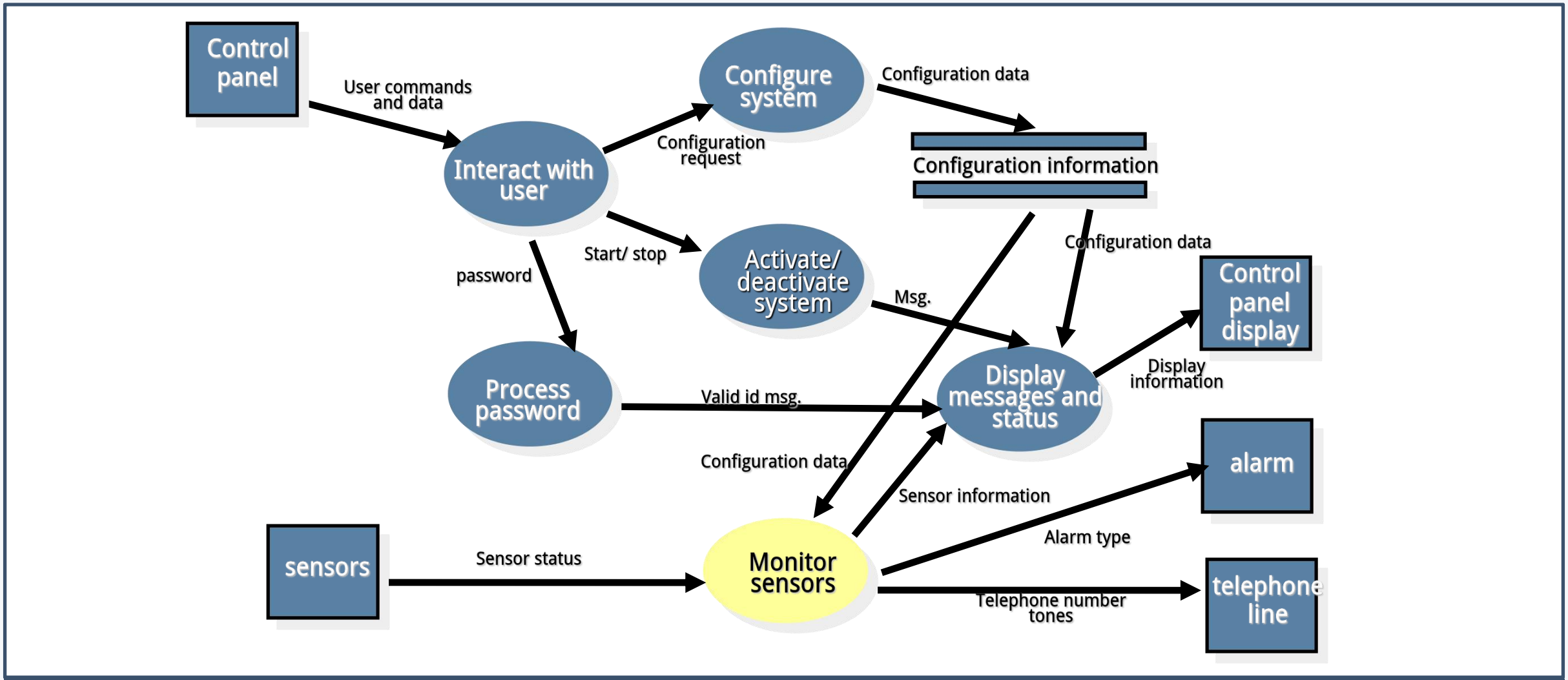


Input Flows
- User commands and data
- Sensor status

Output Flows
- Display information
- Telephone number tones
- alarm (trigger)

Level 1 DFD Example

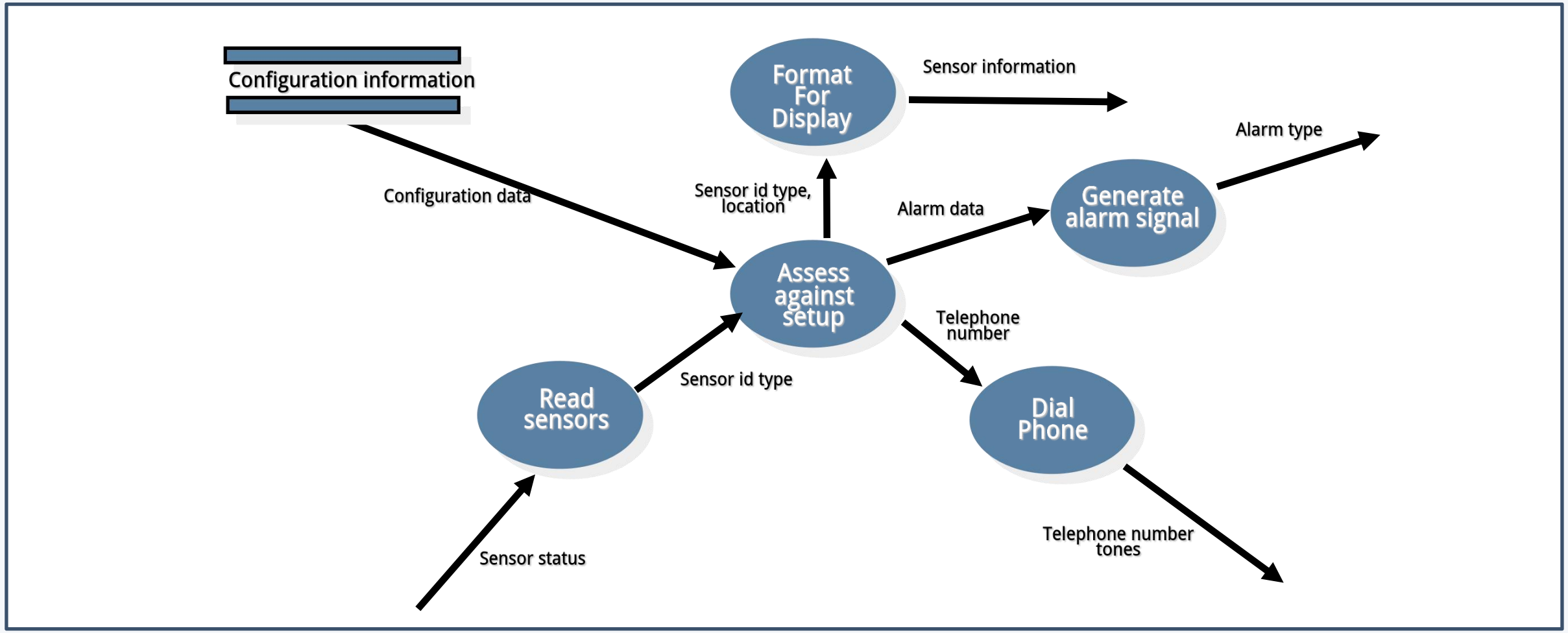
Safe-Home Software Decomposition



The Level 1 DFD shows the **decomposition of Safe-Home software** into six major processes: user interaction, system configuration, activation/deactivation, information display, password processing, and sensor monitoring.

Level 2 DFD Example

Sensor Monitoring Decomposition



Format For Display	Generate Alarm	Assess Against Setup	Read Sensors	Dial Phone
Formats sensor data	Creates alarm signal	Compares to configuration	Gets sensor data	Calls emergency

Process Specification (PSPEC)

Describing Process Logic

PSPEC

A **Process Specification** (PSPEC) describes the processing logic for each bubble in a DFD.

For Each Bubble

Each process (bubble) in a DFD must have a corresponding PSPEC describing its logic.

Implementation-Independent

Describes what the process does, not how it's implemented.

PSPEC Components

Narrative

Textual description of the process in natural language, explaining what the process accomplishes.

Pseudocode (PDL)

Structured, program-like logic using Program Design Language to specify processing steps.

Equations

Mathematical formulas and calculations used to transform input data into output data.

Tables

Decision tables showing conditions and corresponding actions.

Diagrams/Charts

Visual representations of logic flow and data relationships.

Key Point: PSPECs are **essential for design**. They provide the detailed logic specification that designers use to create the actual implementation.

DFDs: A Look Ahead

From Analysis Model to Design Model

Analysis to Design Transition

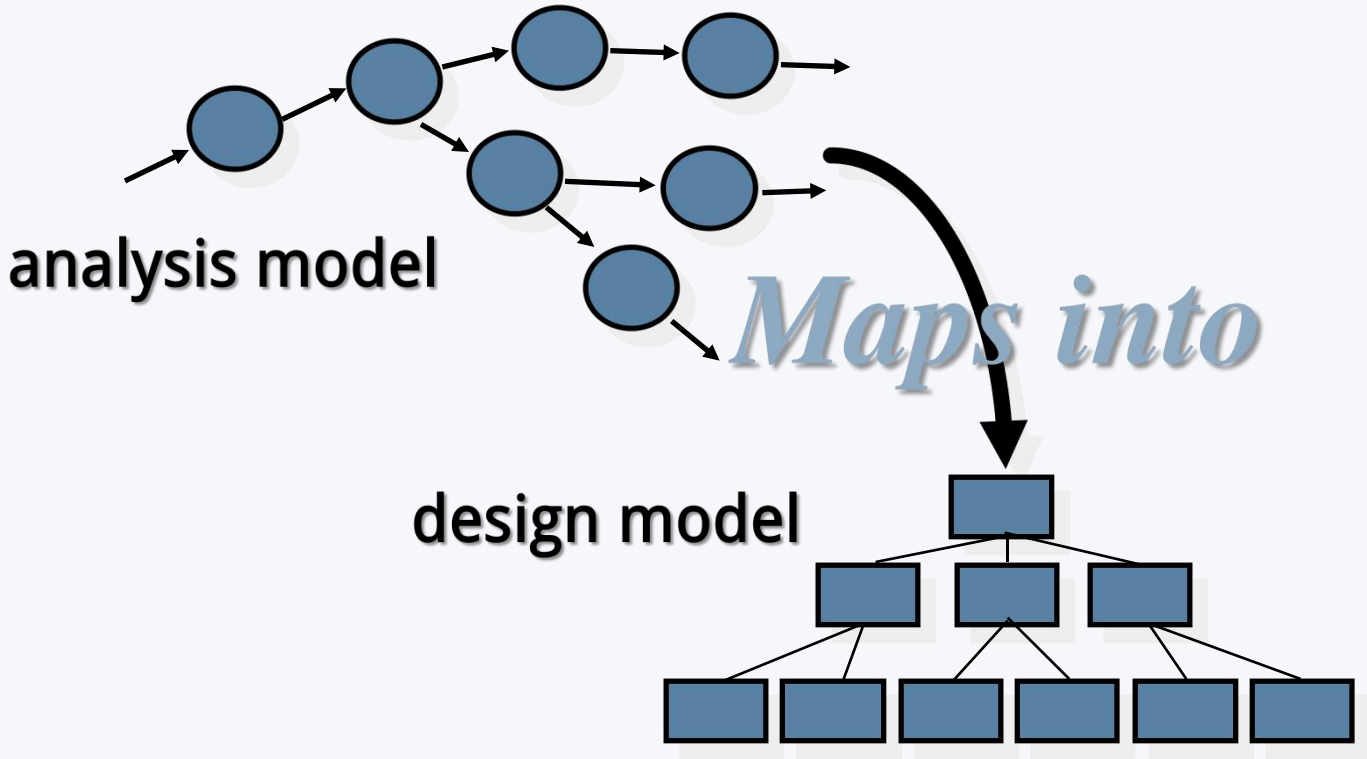
The **analysis model** transforms into a **design model** as development progresses from requirements to implementation.

Analysis Model Focus

- Business requirements
- User needs and goals
- Data and processes
- Functional behavior
- Problem domain understanding

Design Model Focus

- Technical architecture
- Implementation details
- Software components
- System interfaces
- Solution domain structure



Control Flow Diagrams

Events and Process Management

Control Flow

Represents **"events"** and the processes that manage events in the system.

Events

An "event" is a Boolean condition that can be ascertained.

Event Detection

Events can be detected by listing sensors, interrupts, switches, and data conditions.

How to Identify Events

Sensors
List all sensors that are "read" by the software.

Interrupts
List all interrupt conditions that can occur.

Switches
List all "switches" actuated by an operator.

Data Conditions
List all data conditions that trigger events.

Control Items

Review the noun/verb parse from the processing narrative to identify possible control items that could be CSPEC inputs/outputs.

The Control Model

Superimposing Control Flow on DFD

Control Flow Diagram

The control flow diagram is **"superimposed"** on the DFD and shows events that control the processes noted in the DFD.

Dashed Arrows

Control flows (events and control items) are noted by dashed arrows .

Control Spec (CSPEC)

A vertical bar implies an input to or output from a control spec (CSPEC).

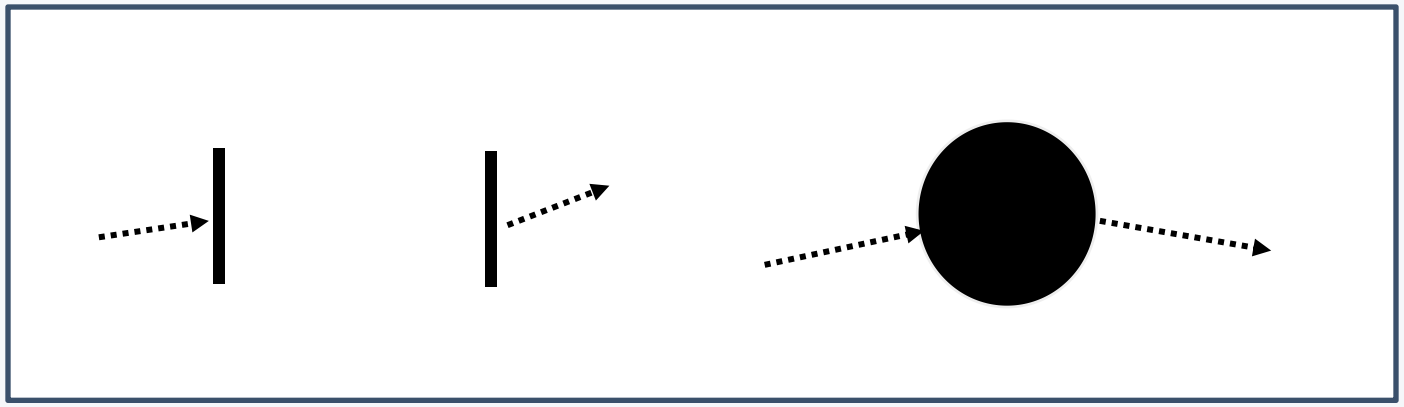
CSPEC Inputs/Outputs

A dashed arrow entering a vertical bar is an input to the CSPEC.

A dashed arrow leaving a process implies a data condition

A dashed arrow entering a process implies a control input read directly by the process

Control Flow Notation

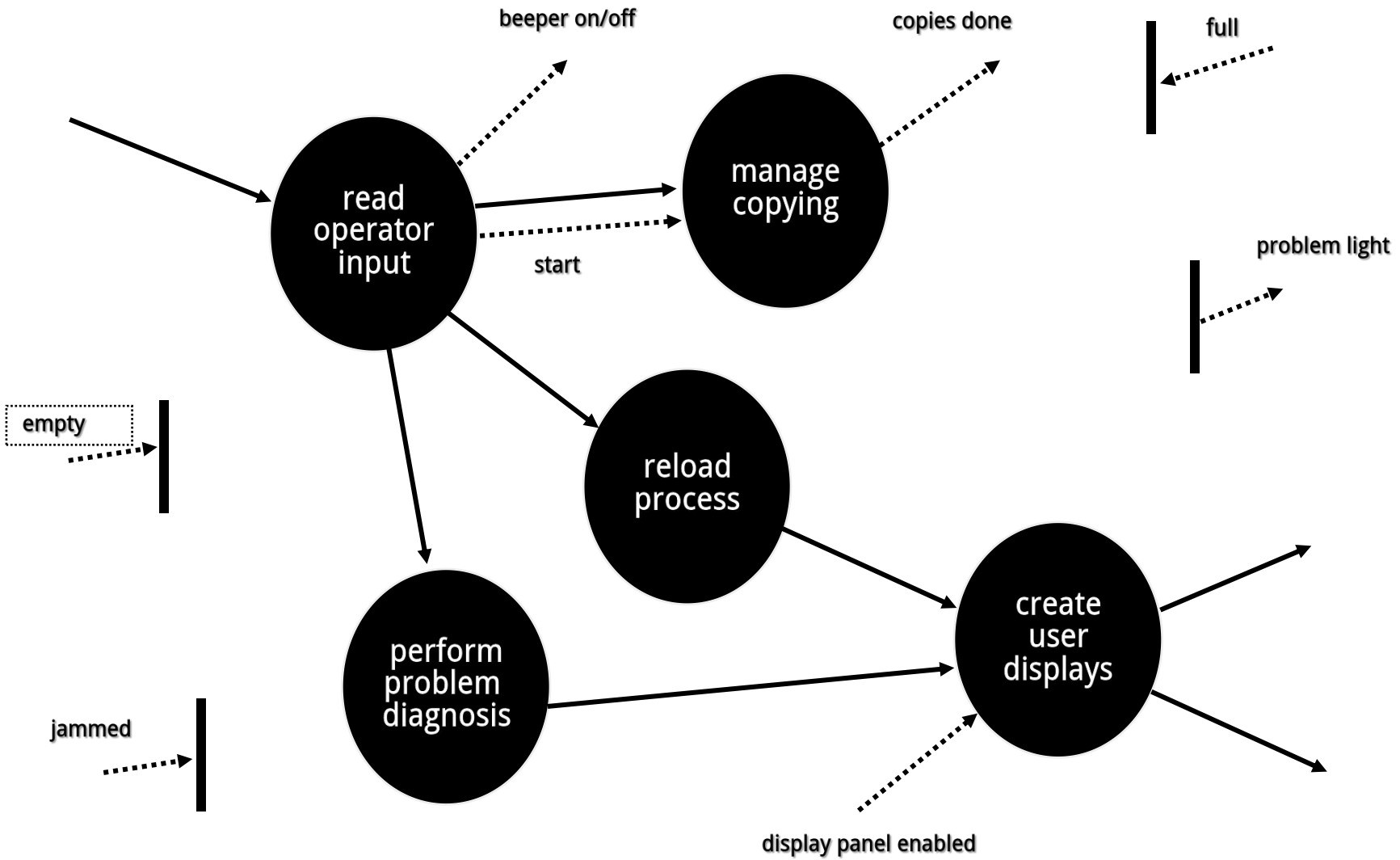


Key Points

- Control flows do not physically activate/deactivate processes
- Activation is done via the CSPEC logic
- Dashed arrows show event triggers and control signals
- Solid arrows show data flow as usual

Control Flow Diagram

Copy Machine Example



Scenario-Based Modeling

Use-Case Driven Approach

Scenario-Based

Focuses on **user scenarios** that describe threads of usage for a system.

Use-Case

A scenario that describes a "thread of usage" for a system.

Actors

Actors represent roles people or devices play as the system functions.

Key Concepts

Actors

Actors are roles played by users or external systems when interacting with the system.

Important: Users can play multiple roles for different scenarios.

Scenarios

Scenarios describe sequences of interactions between actors and the system to achieve a specific goal.

Goals

Each use-case has a clear goal that provides value to the actor(s) involved.

Benefits

Scenario-based modeling helps ensure the system meets **real user needs** and provides **measurable business value**.

Developing a Use-Case

Key Questions to Guide Development

1

Main Tasks or Functions

What are the main tasks or functions that are performed by the actor?

2

System Information

What system information will the actor acquire, produce, or change?

3

External Environment Changes

Will the actor have to inform the system about changes in the external environment?

4

Information Desired

What information does the actor desire from the system?

5

Unexpected Changes

Does the actor wish to be informed about unexpected changes ?

Complete Coverage

Answering these questions ensures comprehensive coverage of actor-system interactions, including normal and exceptional scenarios.



Tip: Focus on **what the actor wants to achieve**, not how the system works. Keep the use-case centered on actor goals and system responses.

Use-Case Diagram

Visual Representation of Use-Cases

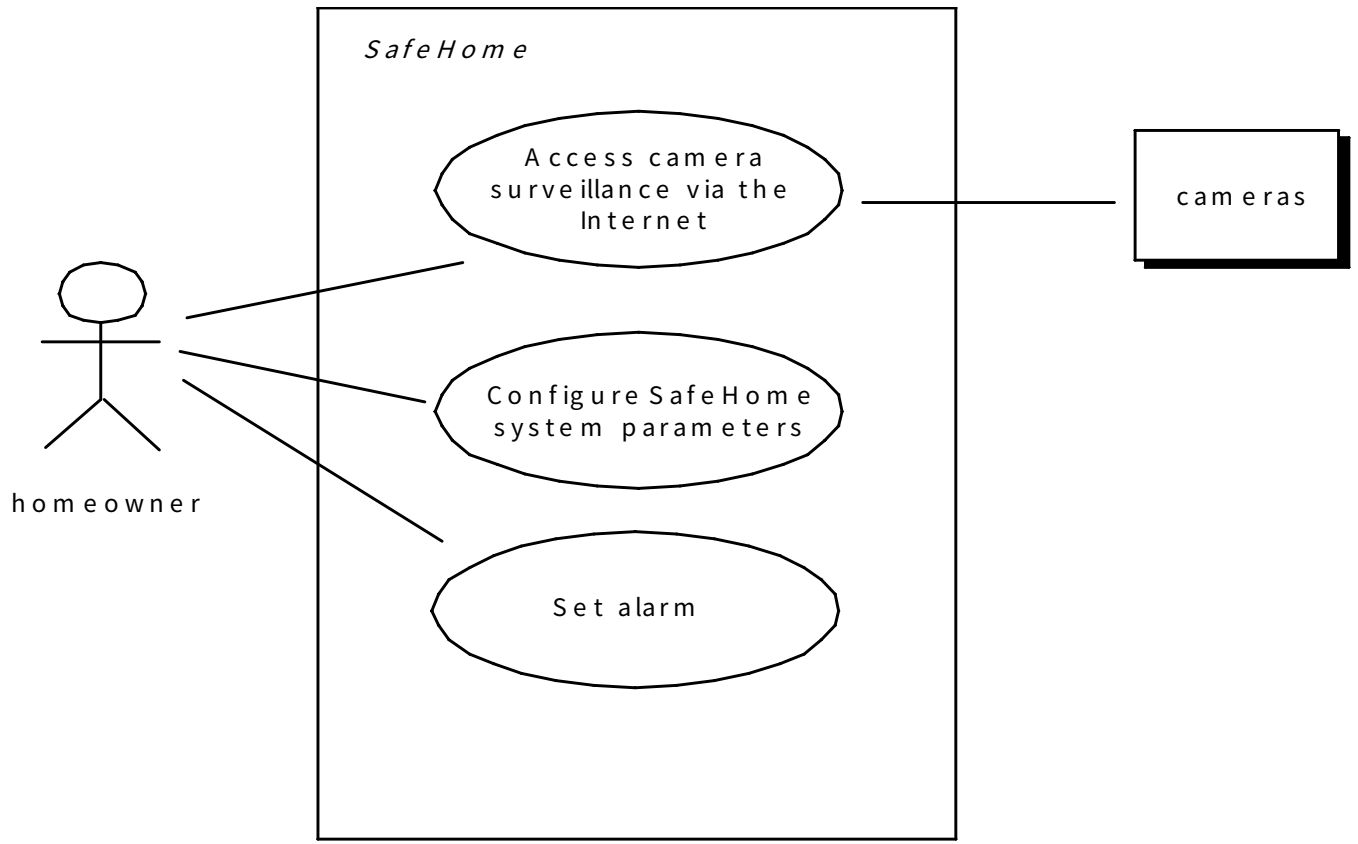


Diagram Elements

- Actor (User Role)
- Use Case
- System Boundary

Key Features

- Shows system scope clearly
- Identifies all actors
- Lists all major functions
- Defines system boundaries

Relationships

Lines between actors and use-cases represent **associations** - interactions between actors and system functions.

ACS-DCV Use Case

Access Camera Surveillance - Display Camera Views

ACS-DCV

Access Camera Surveillance - Display Camera Views

A use case for remote surveillance system access allowing homeowners to view camera feeds via the internet.

Primary Actor

Homeowner - wants to monitor their property remotely.

Goal

To view camera output placed throughout the house from any remote location.

Use Case Development

Key Questions for ACS-DCV

1. How does the homeowner access the system?
2. What authentication is required?
3. How are cameras selected and viewed?
4. What error conditions can occur?

Main Success Scenario

- Log on to Safehome website
- Enter user ID and passwords
- Select "surveillance" function
- Select camera from floor plan
- View live video at 1 frame/sec

Value Proposition

Provides peace of mind and **remote monitoring capability** for homeowners.

ACS-DCV Description

Detailed Use Case Scenario

Use Case: access camera surveillance-display

Actor: homeowner

Main Success Scenario

1. The homeowner logs on to the Safehome Products web site
2. The homeowner enters his or her user ID
3. The homeowner enters two passwords (each at least 8 char)
4. The system displays all major function buttons
5. Selects "surveillance" from major function buttons
6. The homeowner selects "pick a camera"
7. The system displays the floor plan of the house
8. The homeowner selects a camera icon from the floor plan
9. The homeowner selects the "View" button
10. System displays viewing window identified by camera ID
11. System displays video output at one frame per second

Key Observations

Preconditions

- System must be fully configured
- User ID and passwords must be obtained
- Floor plan must be configured

Postconditions

- Camera view is displayed
- Video streaming at 1 fps
- User can monitor remotely

Quality Requirements

- Response time: < 3 seconds
- Video quality: 1 fps minimum
- Security: encrypted connection

Considering Step 6 and 7

Alternative Paths and Exception Analysis

1 Alternative Actions?

Can the actor take some **other action** at this point?

Possible Answers:
- Select "View all cameras"
- Go back to main menu
- Log out of system
- Change configuration

2 Error Conditions?

Is it possible that the actor will encounter some **error condition** at this point?

Possible Errors:
- Floor plan not available
- No cameras configured
- Network connection failed
- Camera not responding

3 Other Behavior?

Is it possible that the actor will encounter some **other behavior** at this point?

Other Behaviors:
- System displays warning
- Automatic camera detection
- Thumbnail preview appears
- Multiple floor plan options

Importance of Question Analysis

Asking these questions helps identify **alternative flows**, **exception scenarios**, and **error conditions** that must be handled in the use case.

Use case Template

ACS-DCV Use Case Header Information

Use Case Header

Use Case: access camera surveillance-display (ACS-DCV)

Primary Actor: homeowner

Goal in context: To view output of camera placed throughout the house from any remote location via Internet

Preconditions

- System must be fully configured
- Appropriate User ID and Pwd must be obtained
- Floor plan must be configured with camera icons

Trigger

The homeowner decides to take a look inside the house while away

Use Case Scenario

Scenario:

1. The homeowner logs on to the Safehome Products web site
2. The homeowner enters his or her user ID
3. The homeowner enters two passwords (each at least 8 char)
4. The system displays all major function buttons
5. The homeowner selects "surveillance" from major function buttons
6. The homeowner selects "pick a camera"
7. The system displays the floor plan of the house
8. The homeowner selects a camera icon from the floor plan
9. The homeowner selects the "View" button
10. System displays viewing window identified by camera ID
11. System displays video output at one frame per second

Use case Template

ACS-DCV Exception Handling

Exceptions

3a. ID or PWD are incorrect or not recognized — see use case: **"Validate ID and passwords"**

5a. Surveillance function not configured — system displays error message; see use case: "Configure surveillance function"

5b. Homeowner selects "View thumbnail snapshots for all cameras" – see use case "View thumbnail snapshots for all cameras"

7a. A floor plan is not available or has not been configured – displays error message and see use case "Configure floor plan"

11a. An alarm condition is encountered –see use case "alarm condition encountered"

Robust Use Case Design

Comprehensive exception handling ensures the system behaves predictably and gracefully when things go wrong.

Use case Template

ACS-DCV Priority and Open Issues

Priority & Timing

Development Priority

Priority: Moderate priority, to be implemented after basic functions

When available: Third increment

Frequency: Infrequent

Communication Channels

Channel to actor: Via PC-based browser and internet connection to Safehome website

Secondary actors: System administrator, camera

Channels to secondary actors:

1. System administrator: PC-based system
2. Camera: wireless connectivity

Open Issues

1. What mechanisms protect unauthorized use by employees?
2. Is security sufficient? Hacking represents major privacy invasion.
3. Will system response via internet be acceptable given bandwidth?
4. Will capability protect video at higher fps when high bandwidth available?

Issue Management

These open issues require **stakeholder discussion** and **technical analysis** before final implementation.

Activity Diagram

Procedural Flow Representation

Activity Diagram

Supplements the use-case by providing a **diagrammatic representation of procedural flow.**

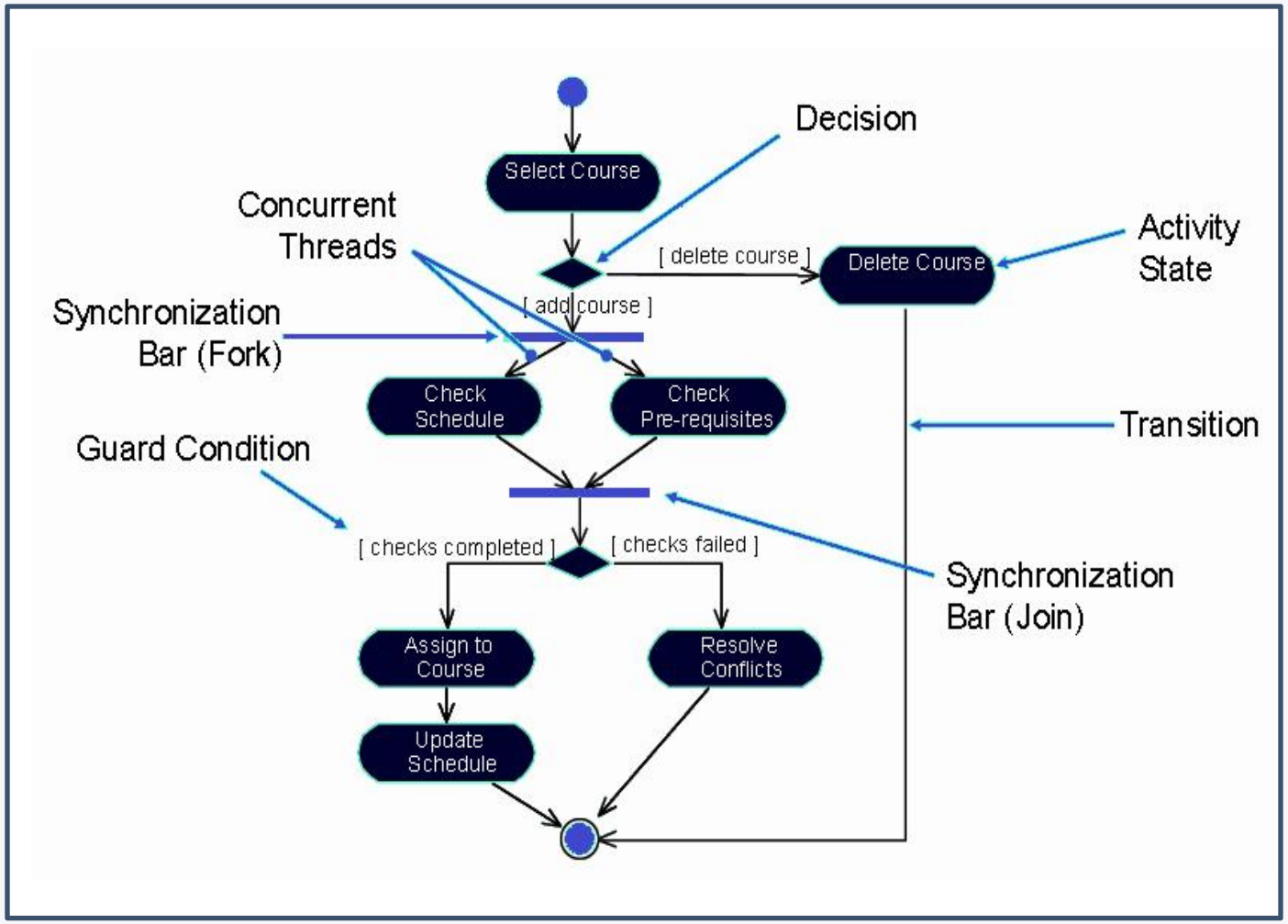
Flowchart-Style

Shows activities, decisions, and control flows in a visual, flowchart-style notation.

UML Notation

Uses UML activity diagram notation with actions, decisions, forks, and joins.

Activity Diagram Elements

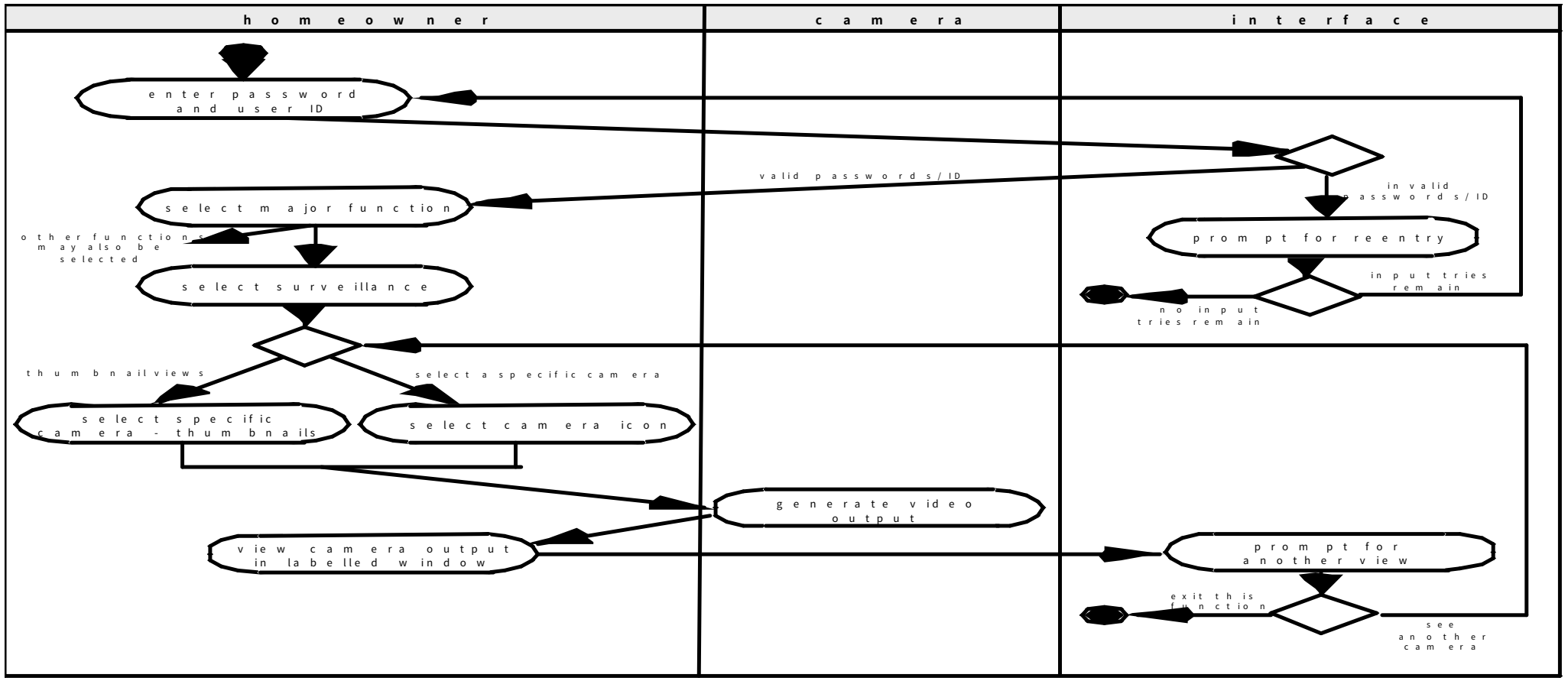


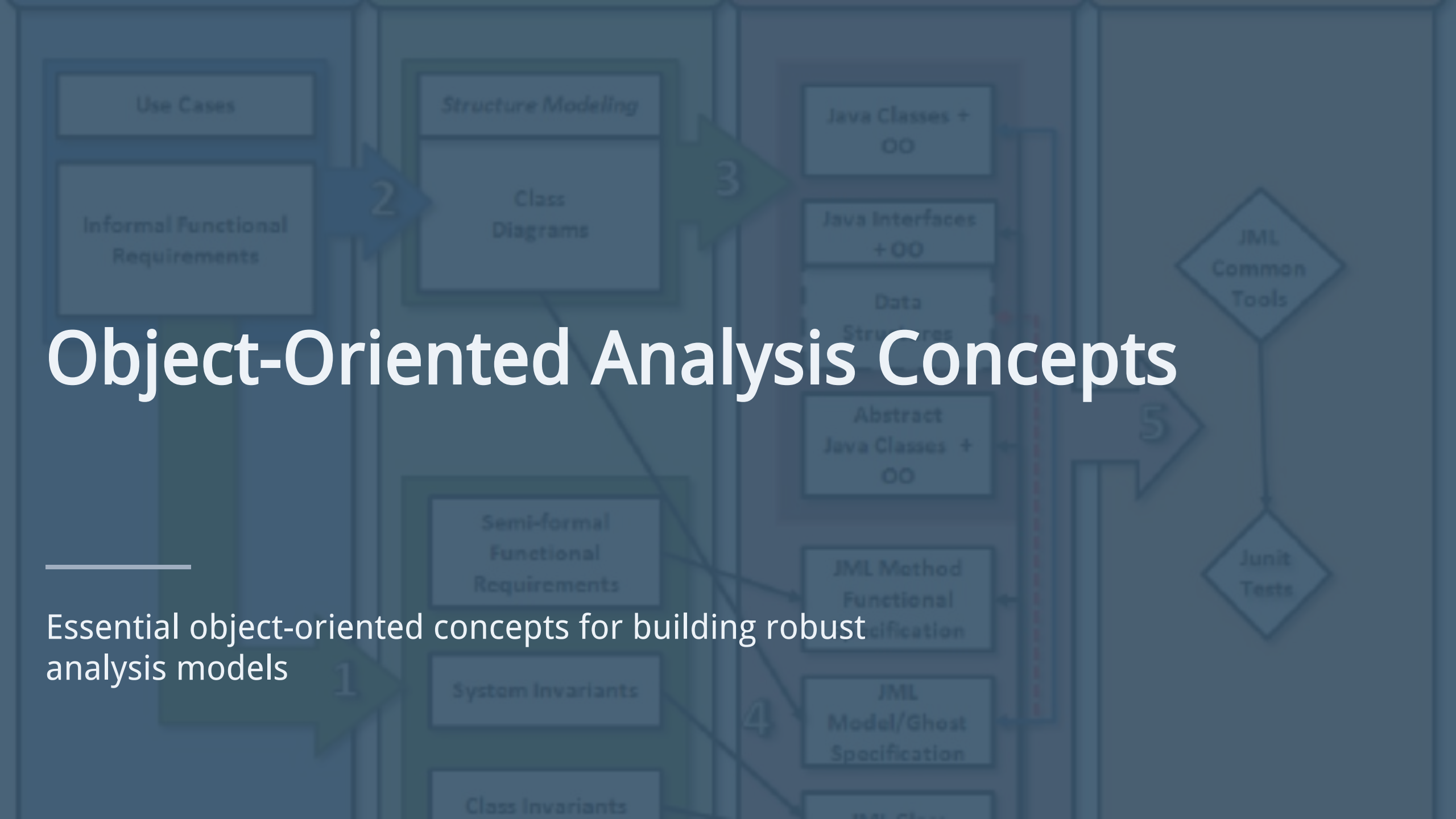
Swimlane Diagrams

Definition

Swimlane diagrams allow the modeler to represent the **flow of activities** described by the use-case.

They indicate which actor or analysis class has responsibility for actions described by activity rectangles.





Object-Oriented Analysis Concepts

Essential object-oriented concepts for building robust analysis models

Object-Oriented Analysis

Core OO Concepts

Foundation Concepts

These concepts must be understood to apply **class-based elements** of the analysis model effectively.

Classes and Objects

Classes are templates defining structure and behavior. **Objects** are instances of classes with specific values.

Example: "Car" class → "Toyota Camry" object

Attributes and Operations

Attributes define data properties. **Operations** (methods) define behaviors and functions.

Attributes: color, speed. Operations: start(), stop()

Encapsulation

Encapsulation bundles data and methods. **Instantiation** creates objects from classes.

Internal state hidden, public methods provide access

Inheritance

Inheritance allows classes to inherit properties and methods from parent classes.

"SUV" inherits from "Car" class

Object-Oriented Analysis

Classes: Definition and Structure

Class Definition

A **class** encapsulates the data and procedural abstractions required to describe the content and behavior of some real-world entity.
It serves as a template for creating objects with common characteristics and behaviors.

Class Components

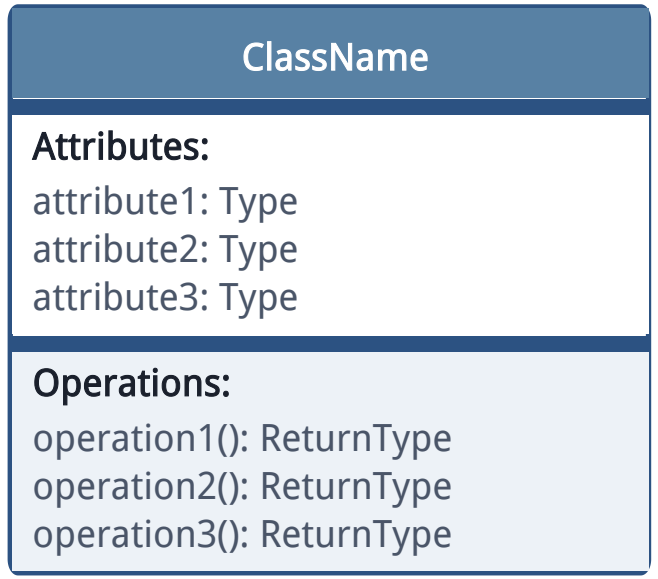
Attributes

A collection of data values that **describe a class** .
Example: name, age, address

Operations

Also called **methods and services** , define behaviors.
Example: calculateSalary(), saveRecord()

UML Class Notation



Related Concepts

Objects

Instances of a specific class with concrete values.

Superclass

Also called a base class, is a generalization of related classes.

Object-Oriented Analysis

Building a Class

Example: Book Class

Book

Attributes:

title: String
author: String
isbn: String
publishedYear: Integer
available: Boolean

Operations:

checkOut(): void
returnBook(): void
getDetails(): String
isAvailable(): Boolean

Example: Vehicle Class

Vehicle

Attributes:

make: String
model: String
year: Integer
vin: String
mileage: Double

Operations:

startEngine(): void
stopEngine(): void
drive(miles: Double): void
getMileage(): Double

Example: Student Class

Student

Attributes:

studentId: String
name: String
email: String
gpa: Double
enrolled: Boolean

Operations:

enroll(course: Course): void
drop(course: Course): void
updateGPA(): void
getTranscript(): String

Class Construction Checklist

- Descriptive class name
- Relevant attributes
- Meaningful operations
- Proper UML notation

Object-Oriented Analysis

What is a Class? Categories

Identifying Classes

Classes can be identified by examining the **problem statement** and looking for these seven categories of real-world entities.

Occurrences

Events that occur within the context of system operation.

Examples:
Property transfer, robot movement, transaction

Roles

Roles played by people who interact with the system.

Examples:
Manager, engineer, salesperson, customer

Organizational Units

Groups or teams relevant to an application.

Examples:
Division, department, team, project group

Things

Physical objects in the information domain.

Examples:
Reports, displays, letters, signals

Places

Locations that establish context of the problem. Examples: Manufacturing floor, warehouse, office

Examples:
manufacturing floor or loading dock

External Entities

Systems, devices, or people that produce/consume information.

Examples:
Other systems, sensors, external databases

Structures

Architectural elements that define related classes of objects.

Examples:
Sensors, four-wheeled vehicles, computers, mechanical assemblies

Object-Oriented Analysis

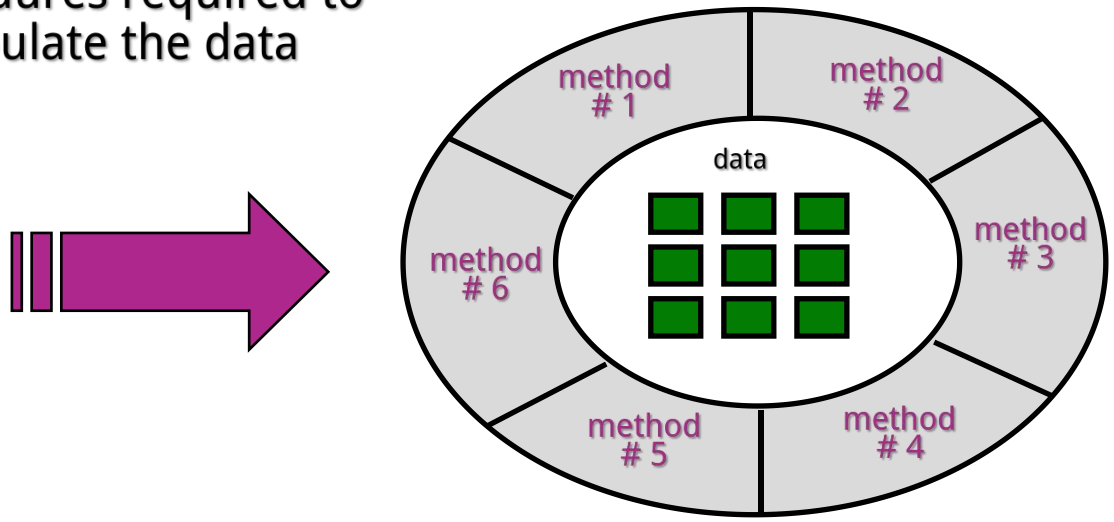
Encapsulation & Information Hiding

Core Principle

The object **encapsulates both data and the logical procedures** required to manipulate the data, achieving "information hiding".

Encapsulation in Action

The object encapsulates both data and the logical procedures required to manipulate the data



Achieves “information hiding”

Information Hiding

Benefits:

- **Security:** Prevents unauthorized access
- **Modularity:** Independent development
- **Maintainability:** Internal changes don't affect clients
- **Flexibility:** Easy to modify implementation

Access Levels

Pr	Private - class only
Pr	Protected - subclasses
Pu	Public - any object

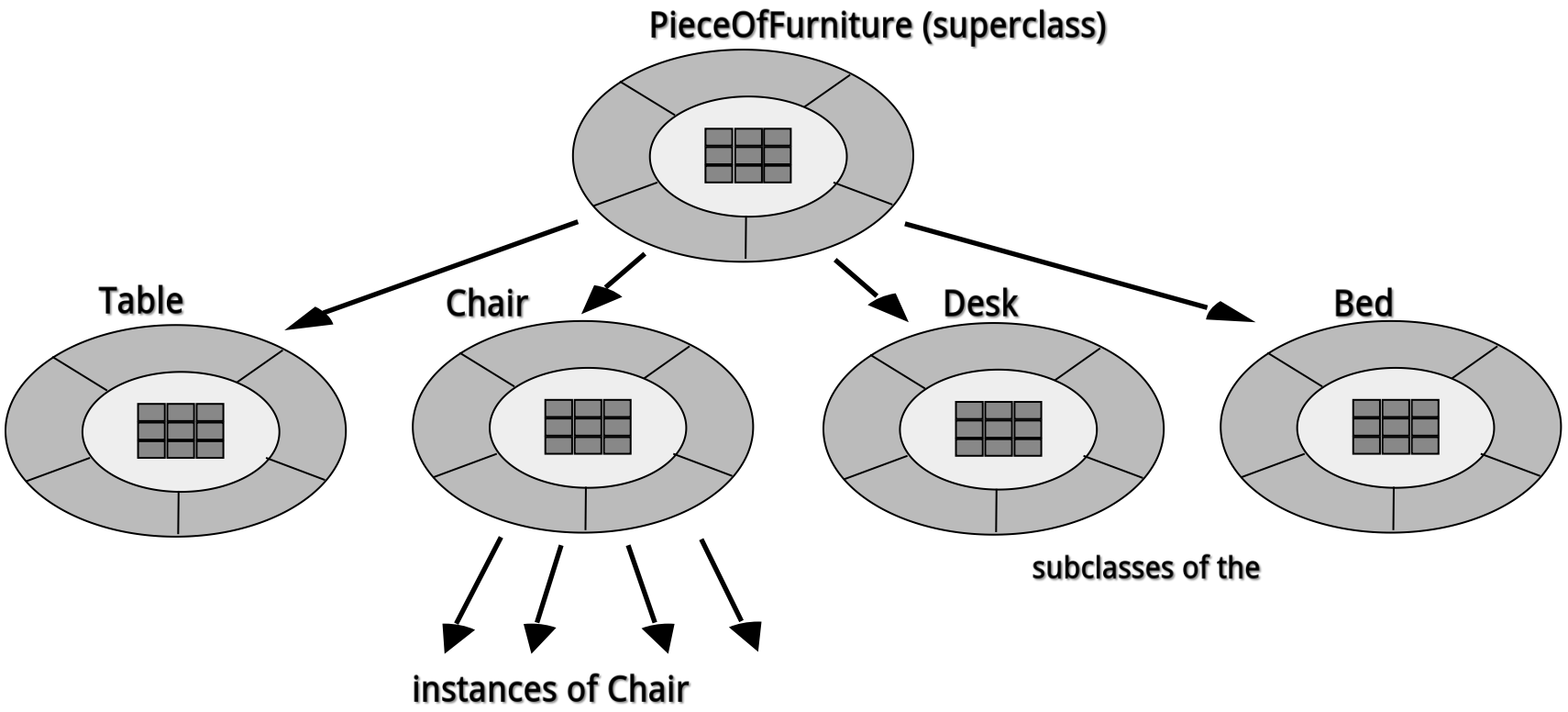
Object-Oriented Analysis

Class Hierarchy & Inheritance

Inheritance Structure

Inheritance creates a **"is a kind of"** relationship, enabling code reuse and specialization.

Furniture Class Hierarchy Example



Generalization

Superclass is a **generalization** of subclasses, containing common features.

Benefits:
- Code reuse
- Easier maintenance
- Polymorphism support

Specialization

Subclasses can **add new features** or **override** inherited ones.

Example: Chair adds hasArmrest, Desk adds hasDrawers

Object-Oriented Analysis

Methods (Operations & Services)

Definition

A **method** is an executable procedure that is encapsulated in a class and is designed to operate on one or more data attributes defined as part of the class.

Methods define the behavior and functionality that objects of the class can perform.

Message Passing

A method is **invoked via message passing**. One object sends a message to another object to request a service.

```
bookObject.  getTitle()
Object sending message to method
```

Method Examples

BankAccount Class Methods

- **deposit(amount):** Adds funds
- **withdraw(amount):** Removes funds
- **getBalance():** Returns current balance
- **transfer(toAccount, amount):** Moves funds

Student Class Methods

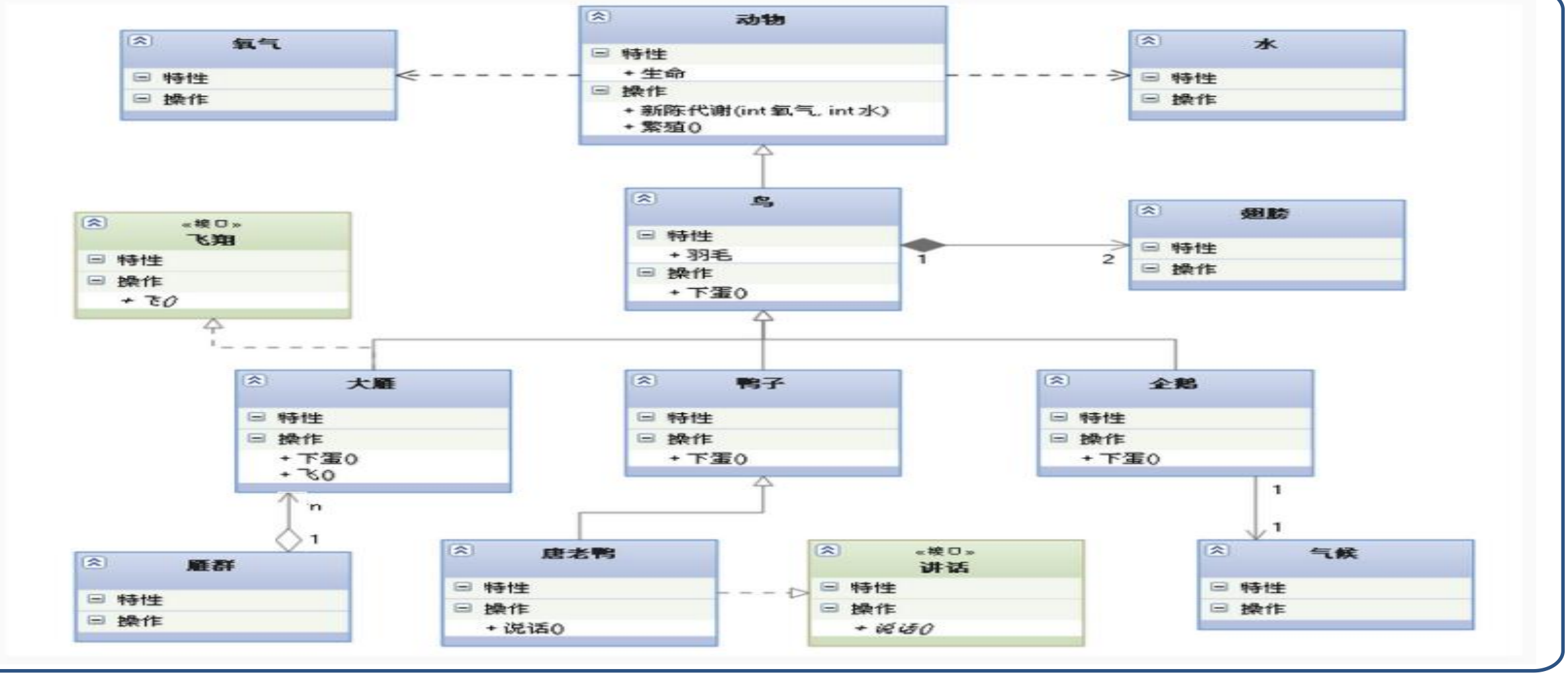
- **enroll(course):** Adds course
- **drop(course):** Removes course
- **calculateGPA():** Computes GPA
- **getTranscript():** Returns record

Method Characteristics

- ✓ Named procedures
- ✓ May have parameters
- ✓ May return values
- ✓ Encapsulated in class
- ✓ Operate on attributes
- ✓ Invoked by messages

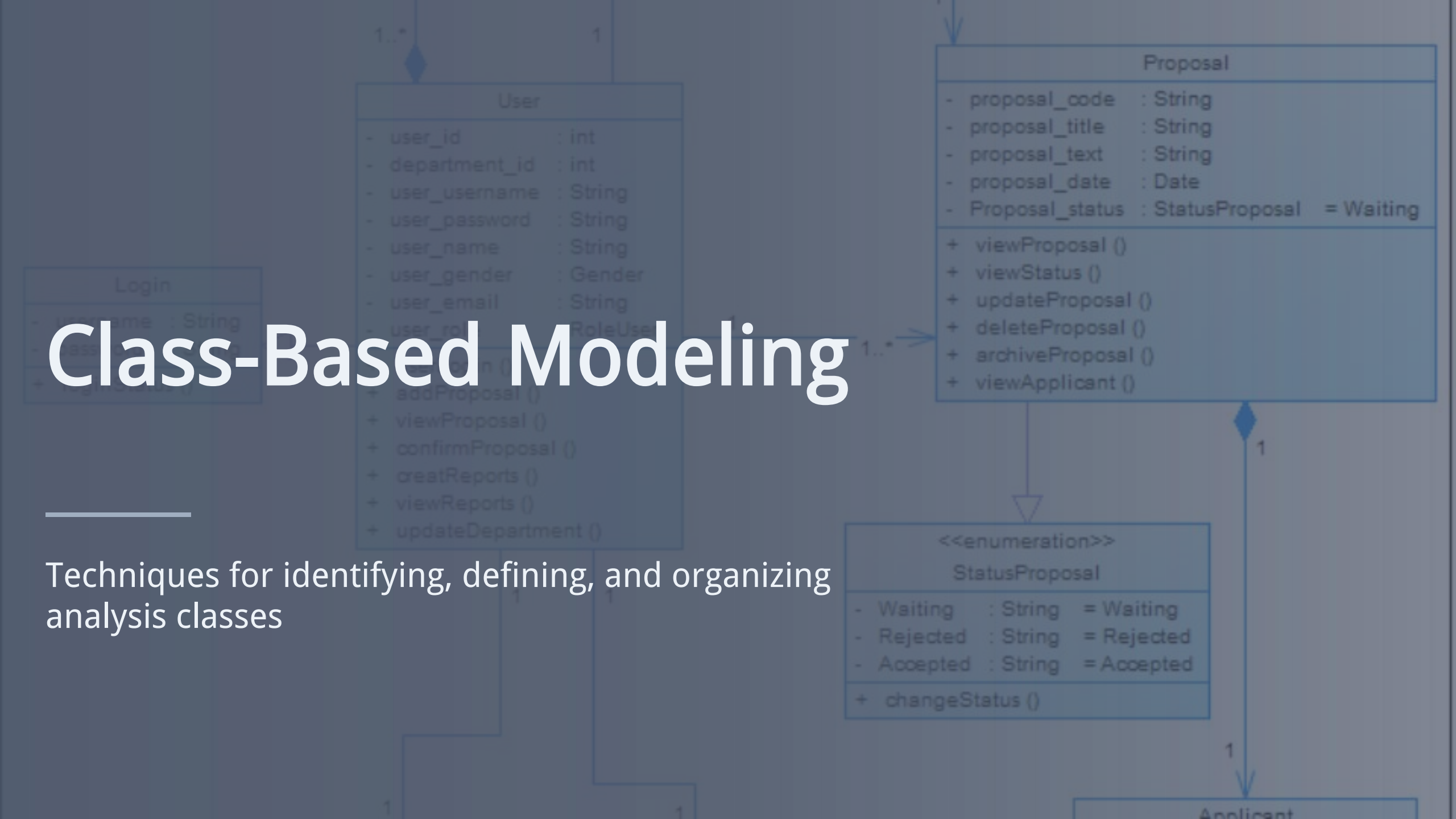
Class Relationship

Generalization > Realization > Composition > Aggregation > Association > Dependency



Class-Based Modeling

Techniques for identifying, defining, and organizing analysis classes



Class-Based Modeling Process

Six-Step Process

A systematic approach to **identifying analysis classes** by examining the problem statement and defining class structure.

1 Identify Conceptual Classes

Extract core domain concepts and define their attributes to build the static structure model.

Domain Focus:
Model real-world concepts, not code constructs.

2 Build Dynamic Model

Describe object interactions and collaborations using sequence diagrams for behavioral clarity.

Strict Order:
Follow the use case flow precisely; no out-of-order messages.

3 Assign Operations

Derive operations from messages and assign operations for domain classes.

Consider:
Distribute responsibilities to prevent a single class from becoming too complex.

Class-Based Modeling Process

Six-Step Process

A systematic approach to **identifying analysis classes** by examining the problem statement and defining class structure.

4 Model States

Map lifecycle behaviors and transitions for complex entities using state machine diagrams.

Reflect Business Rules:
Ensure states represent real-world business logic, not just internal technical flags.

5 Verify & Review

Validate model consistency, completeness, and alignment with stakeholder requirements.

Iterative Process:
OOA is cyclical—expect to revisit and revise earlier models as inconsistencies are identified during verification.

6 Assemble Deliverables

Compile all artifacts into a comprehensive Object-Oriented Analysis (OOA) model.

Foundation for Design:
This package is the critical input for the Object-Oriented Design (OOD) phase, ensuring alignment with business goals.

Step 1: Identify Conceptual Classes & Build Domain Model

Three Class Types

Analysis classes can be organized into **three primary types** based on their architectural role.

Boundary Classes

Used to create the **interface that users see** and interact with.

Characteristics:

- Handle system interaction
- Transform user input
- Present system output

Examples:
Interactive screen, printed report, API

Controller Classes

Manage a **"unit of work"** from start to finish.

Responsibilities:

- Create or update entity objects
- Instantiate boundary objects
- Validate data communication

Examples:
Report generator, workflow manager

Entity Classes

Also called **model or business classes**, extracted directly from the problem statement.

Characteristics:

- Store persistent information
- Represent business entities
- Independent of interfaces

Examples:
FloorPlan, Sensor, Customer, Product

General Identification Rules



Boundary Class: Interaction, Interface, Input & Output

It is only responsible for interacting with users, external devices and external systems, receiving and displaying information. It contains no business logic and does not store business data.



Control Class: Process, Logic, Scheduling & Validation

It encapsulates the complete business processes, rules, data validation, computing logic, state updates and exception handling of a use case. It bridges boundary classes and entity classes, with no interactive behavior and no data persistence.



Entity Class (Core Domain Class): Business Noun, Data Carrier, Persistence Support

It represents real business entities in the system, which need data storage, participate in business calculation and support persistent storage. It constitutes the core domain model of the system.

Identification Procedures 1

Read Use Case Specification and Extract Key Elements

Fully read the use case overview, actors, preconditions, postconditions, basic flow and alternative flows, and mark three types of core information:

- Human-computer interaction behaviors (operation, input, display, printing, pop-up prompt)
- Business logic behaviors (calculation, validation, matching, update, scheduling, process interception)
- Business entity nouns (business objects that need system recording, storage and management)

Identification Procedures 2

Identify Boundary Classes Based on Interactive Behaviors

Scan all business links involving user operation, interface display, device interaction and information prompt:

- Input and operation behaviors → Identify operation interface boundary class
- Result output, printing and pop-up prompts → Identify output & display boundary class
- Core principle: One independent interactive responsibility corresponds to one boundary class without merging or redundancy

Identification Procedures 3

Determine Control Class Based on Business Processes

1. Integrate all non-interactive and non-data logical behaviors: process execution, rule validation, numerical calculation, data update, exception interception and transaction scheduling.
2. A single use case only requires one core control class to schedule the entire business process.
3. The control class acts as a hub: it receives user requests from boundary classes and operates entity class data to complete business closure.

Identification Procedures 4

Extract Entity Classes Based on Business Nouns and Data Requirements

Screen persistent business entities from use case overview, pre/post conditions and business flows, divided into two categories:

1. Basic Entities: core business nouns appearing directly (e.g., commodity, order, promotion activity, operator)
2. Derived Associated Entities: essential entities for resolving many-to-many relationships and storing detailed data (e.g., order item for multi-commodity detailed data of a single order)

Screening criterion: Any business object that requires data storage, participates in calculation, has state changes or needs log recording is a domain entity class

Class Selection Criteria

1. Retained Information
2. Needed Services
3. Multiple Attributes
4. Common Attributes
5. Common Operations
6. Essential Requirements

Identification Procedures 5

Verify Class Integrity via Exception Flows

Verify the rationality and integrity of all classes combined with use case exception/alternative flows:

1. Exception prompts are displayed by boundary classes
2. Exception validation and process interception are handled by control classes
3. Exception triggering conditions depend on data of corresponding entity classes

Identification Key Guidelines

Taboos & Key Guidelines (Error Avoidance)

1. All classes must be derived from use case specifications; arbitrary creation of unmentioned classes is prohibited.
2. Strict responsibility separation: Boundary classes contain no business logic; control classes store no data; entity classes handle no business processes.
3. A single business use case has only one core control class; duplicate creation is forbidden.
4. For scenarios with detailed data and many-to-many associations, derived entities must be split to avoid domain model design defects.

Step 2: Build Dynamic Model

Sequence Diagram Model

A Sequence Diagram is a core UML dynamic model. It describes time-ordered message interactions, calling relationships and execution flows between objects within a single use case.

Principles 1

Single use case principle: one sequence diagram corresponds to only one independent use case, no cross-business or mixed scenarios.

Principles 2

Time-ordered principle: all messages are arranged strictly from top to bottom according to business execution sequence.

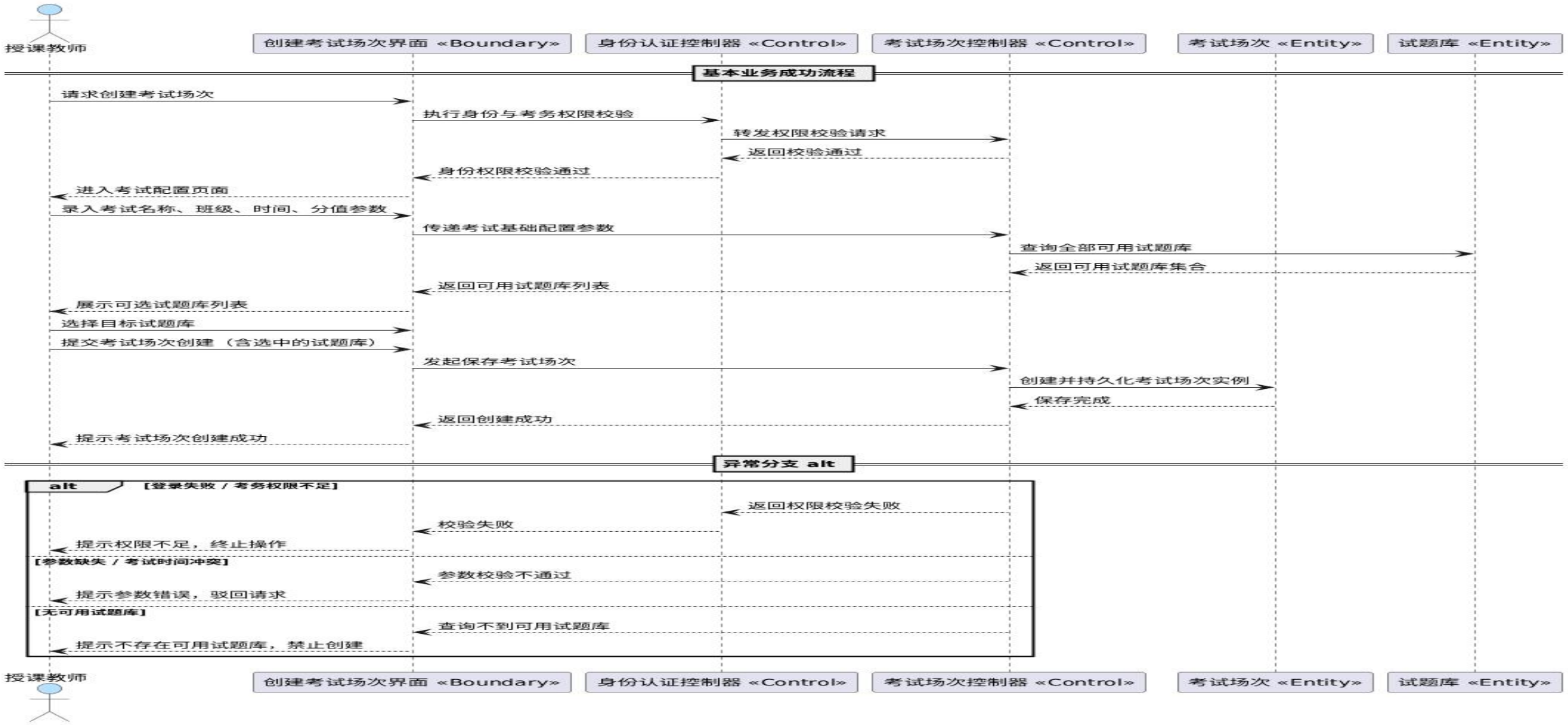
Principles 3

Layered interaction principle: strictly follow the layered call of Actor → Boundary Class → Control Class → Entity Class; cross-layer direct calls are prohibited.

Principles 4

Traceable message principle: all interaction messages are derived from use case specifications, and each message corresponds to a specific business action.

Sequence Diagram



Sequence Diagram Modeling Procedure

1. Confirm Modeling Scope and Lock Single Use Case Scenario

Select one independent and complete business use case as the modeling basis, and read the full use case specification including overview, preconditions, postconditions, basic normal flows and exception extension flows. Clarify the business objectives and execution boundaries of current modeling, ensure modeling for only one closed-loop business process, and avoid mixed multi-business confusion.

2. Extract All Interactive Objects and Actors

Screen all interactive roles and objects from identified domain class models and use cases, classify them by layered standards, and take them as vertical lifelines of the sequence diagram:

1. External Actors: system external roles such as users, administrators and external devices;
2. Boundary Objects: interface interaction classes responsible for request reception and result display;
3. Control Objects: business scheduling core responsible for process validation, calculation and logic control;
4. Entity Objects: data persistence classes responsible for data query, storage and state update.

Ensure no omission, no arbitrary addition and no redundancy.

Sequence Diagram Modeling Procedure

3. Decompose Business Process and Sort Time-Series Message Queue

Decompose use case natural language processes sentence by sentence, convert each business action into standardized object message calls, and sort them in chronological order:

1. Initial action: business request initiated by actors;
2. Intermediate actions: parameter reception, business validation, data query, logic calculation and data update;
3. Closing actions: result return, page display, log recording and process termination.

Distinguish synchronous calls and return messages, sort out complete message links, and ensure full correspondence with use case flows.

4. Arrange Layered Structure and Standardize Message Calling Rules

1. Actors only call boundary classes, without direct access to control classes or entity classes;
2. Boundary classes only receive requests and transmit parameters, only call control classes, and contain no business logic;
3. As the interaction hub, control classes receive requests from boundary classes and call entity classes to complete data operations;
4. Entity classes only respond to calls from control classes, execute CRUD and state updates, and never initiate interactions actively.

All messages are strictly layered horizontally without cross-layer or reverse calls.

Sequence Diagram Modeling Procedure

5. Add Branches, Loops and Exception Flows to Complete Complex Logic

Supplement structured logic components for real business scenarios to form a complete process closed-loop:

1. Conditional branches (opt/alt): handle scenarios such as validation pass/fail, permission existence and sufficient/insufficient inventory;
2. Loop structures (loop): handle repeated businesses such as batch operations and multi-data traversal;
3. Exception interception: handle abnormal scenarios such as business errors, invalid parameters and process termination.

Ensure the sequence diagram covers not only normal main flows, but also all key extension and boundary scenarios.

6. Match Class Diagram Methods and Realize Message-Method Binding

Each message on the sequence diagram shall correspond to a method of the object in the domain class diagram. If such method already exists in the preliminary domain class diagram, bind and reuse it directly. If the method is missing, update the domain class diagram reversely and add this business method to the corresponding class.

1.Boundaryclass messages correspond to interface reception, display and prompt methods;

2.Controlclass messages correspond to business validation, process scheduling and calculationlogic methods;

3.Entityclass messages correspond to data query, modification, update and statechange methods.

Ensure every message is backed by corresponding class methods, achieve full alignment between dynamic flows and static structures, and eliminate arbitrary orphan messages.

Sequence Diagram Modeling Procedure

7. Global Verification to Realize Triple Consistency of Process, Model and Requirement

Complete final verification and eliminate all modeling defects:

1. Correct time sequence: top-down message order fully conforms to business execution logic;
2. Compliant layering: no cross-layer calls, no responsibility crossing and no interaction logic errors;
3. Complete scenarios: full coverage of normal flows, exception branches and loop logic;
4. Model consistency: messages match class methods, and flows fully correspond to use case specifications;
5. No redundancy or omission: no redundant messages or missing steps, ensuring a clean and rigorous model.

Sequence Diagram Modeling Procedure

Common Modeling Errors

1. One diagram cannot correspond to multiple use cases; redundant or missing lifeline objects are prohibited.
2. Cross-layer direct connection is prohibited: actors cannot directly call entity classes, which must be transferred via boundary and control classes.
3. Time sequence chaos is prohibited: message order must strictly match the chronological business logic.
4. Pure main flows without exception branches are prohibited, which will cause incomplete scenario coverage.
5. Arbitrary messages are prohibited; all interactions must be traceable to use cases and domain class methods.

Step 3: Assign Operations

Assign Operations for domain classes

Derive Operations from Sequence Diagrams

Each message sent in a sequence diagram translates directly to an operation that the receiving object must implement. Collect all such messages to define the class interface.

Update the Domain Class Diagram

Add the derived operations as methods to their respective classes. Ensure the signatures (parameters, return types) align with the interaction logic.

Key Considerations

Abstract Business Logic
Operations represent "what" needs to be done, not "how" to implement it.

Avoid "God Classes"
Distribute responsibilities to prevent a single class from becoming too complex.

Iterative Refinement:
If inconsistencies arise between the class model and sequence diagrams, revise the model or the interaction logic.

Step 4: Model States

States Diagram

A State Diagram is a core UML dynamic model. It describes the complete lifecycle, discrete states, triggering events, guard conditions and transition behaviors of a single complex domain entity from creation to destruction. It focuses on internal state changes of a single object rather than multi-object interactions, and complements sequence diagrams to perfect the OOA model.

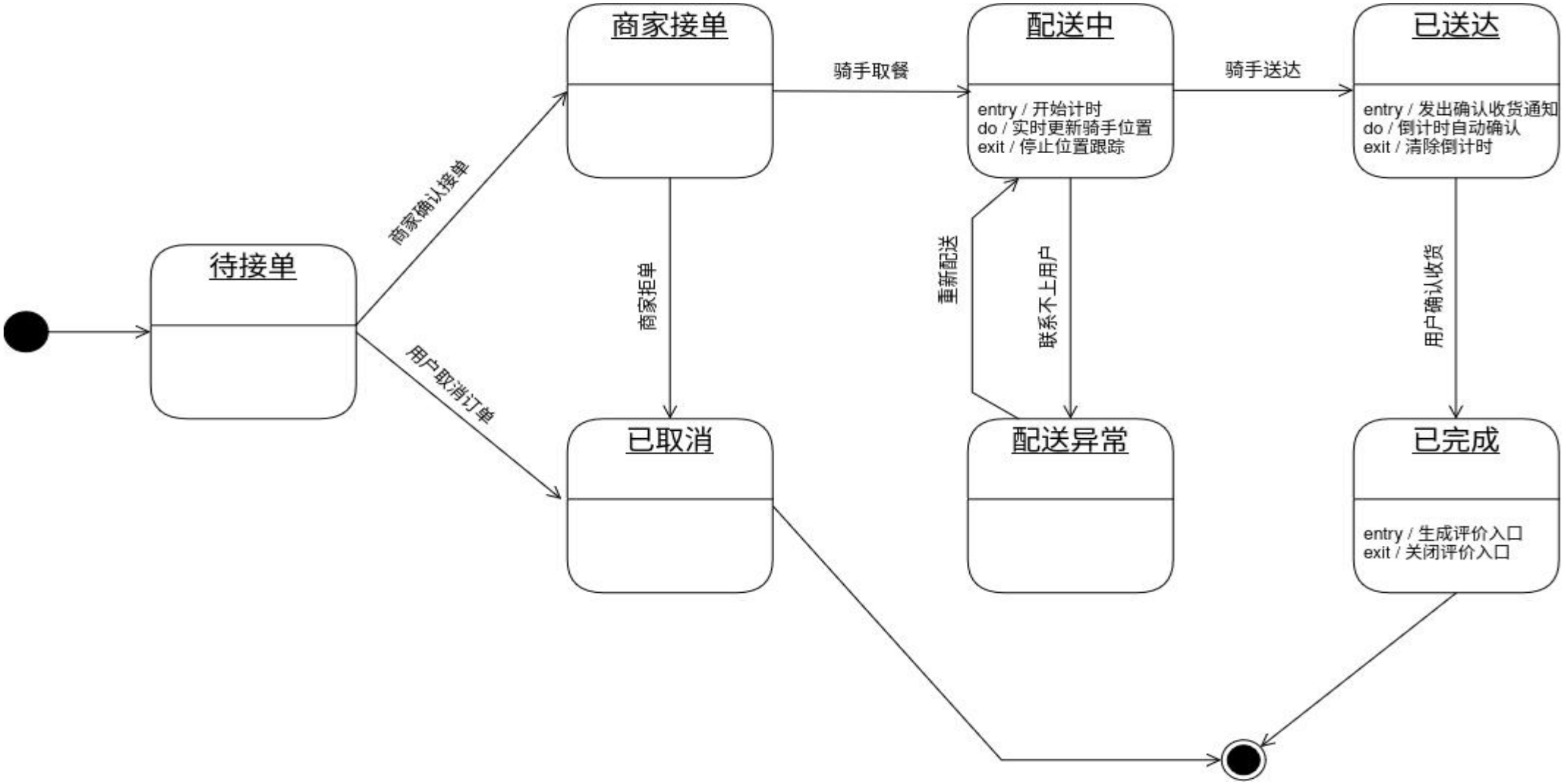
•Single Entity Principle: One state machine diagram corresponds to only one complex core entity, no cross-object or cross-business modeling.

Mutually Exclusive State Principle: An entity stays in only one state at any time; all states are independent and non-overlapping.

•Event-Driven Principle: All state transitions must be triggered by explicit business events; no arbitrary state jumps.

•Traceability Principle: All states, events and behaviors are derived from use case flows and dynamic interaction scenarios.

Step 4: Model States



State Diagram Modeling Procedure

1: Select Complex Entity and Define Lifecycle Boundary

Select state-sensitive, lifecycle-complete and process-driven complex entities (e.g., order, borrowing record, payment record) from domain classes. Simple entities such as users and commodities do not require state modeling. Define the lifecycle start (instantiation/initial state) and end (completion/archiving/destruction) to clarify modeling boundaries.

2: Enumerate and Define All Valid Discrete States

Based on use case specifications, sequence diagram flows and entity attributes, enumerate all valid entity states and classify them into initial state, intermediate active states and final termination state. Define each state with standardized business description to achieve full coverage, no omission, no redundancy and no overlap, and eliminate all illegal states.

State Diagram Modeling Procedure

3: Extract All Triggering Events and Transition Preconditions

Extract all state-changing business trigger events from dynamic model interactions, including user operations, system validation, approval actions, payment completion and timeout invalidation. Define exclusive trigger events and preconditions for each transition to ensure "no transition without valid event and evidence".

4: Build State-Event-Target Transition Links

Follow the core formula: Current State + Trigger Event = Target State to establish all transition relationships. Start from the initial state and connect all intermediate states in sequence, distinguish normal and exception branches, and build a complete closed-loop state link. Forbid illegal jumps, reverse transitions, state gaps and dead loops.

State Diagram Modeling Procedure

5: Add Guard Conditions and Exception Branch Logic

For complex multi-branch scenarios, add Guard Conditions to state transitions to distinguish different results under the same trigger event. Supplement abnormal transitions such as failure, rejection, timeout and parameter errors to ensure the state machine covers both normal business flows and all exception and boundary scenarios.

6: Bind Lifecycle Behaviors and Associate Domain Class Methods

Bind system behaviors to each state and transition, including entry behavior, do behavior, exit behavior and transition behavior. Associate all state actions with corresponding methods of entity classes and control classes to ensure executable state changes, and realize full alignment among state machine diagrams, sequence diagrams and class diagrams.

State Diagram Modeling Procedure

7: Globally Verify State Machine Integrity and Logical Self-Consistency

Complete four-dimensional verification:
1. State integrity: no lifecycle breaks or missing states;
2. Valid events: all triggers are derived from real business scenarios;
3. Logical fluency: no conflicting transitions or contradictory branches;
4. Model consistency: all state changes fully match dynamic interactions, business rules and class methods.

Common Modeling Errors

- Multi-entity modeling is prohibited: one state diagram only describes the lifecycle of one single entity.
- Overlapping states are prohibited: an entity has only one exclusive state at a single moment.
- Untriggered jumps are prohibited: all transitions must be marked with clear events or guard conditions.
- Model confusion is prohibited: state diagrams only describe internal entity state changes, not object message interactions.
- Open lifecycle is prohibited: the diagram must contain initial and final states to form a closed and converged process.

Step 5: Model Consistency Verification & Review

Model Verification & Review

Cross-Model Checks

Verify alignment between sequence, class, and state diagrams to ensure logical consistency across all views.

Stakeholder Review

Validate the model with business owners to confirm it accurately reflects real-world needs and core processes.

Refine & Simplify

Eliminate redundancy and unnecessary complexity to keep the analysis model focused and easily understandable.

Iterative Process
OOA is cyclical—expect to revisit and revise earlier models as inconsistencies are identified during verification.

Avoid Implementation Details
Do not include database schemas, UI specifics, or framework code—focus on the "what," not the "how."

Separate Concerns
Analysis answers "What is the system supposed to do?" while design answers "How to build it." Keep them distinct.

Step 6: Assemble Complete OOA Deliverables

Deliverable package consolidates all analytical artifacts into a cohesive model

Use Case Model

Diagrams, textual specs, and a unified business glossary defining user-system interactions.

Static Structure

Conceptual class diagrams detailing entities, attributes, associations, and operations.

Dynamic Interactions

Sequence diagrams mapping core use case flows and object collaborations over time.

State Dynamics

State machine diagrams for entities with complex lifecycles and behavioral transitions.

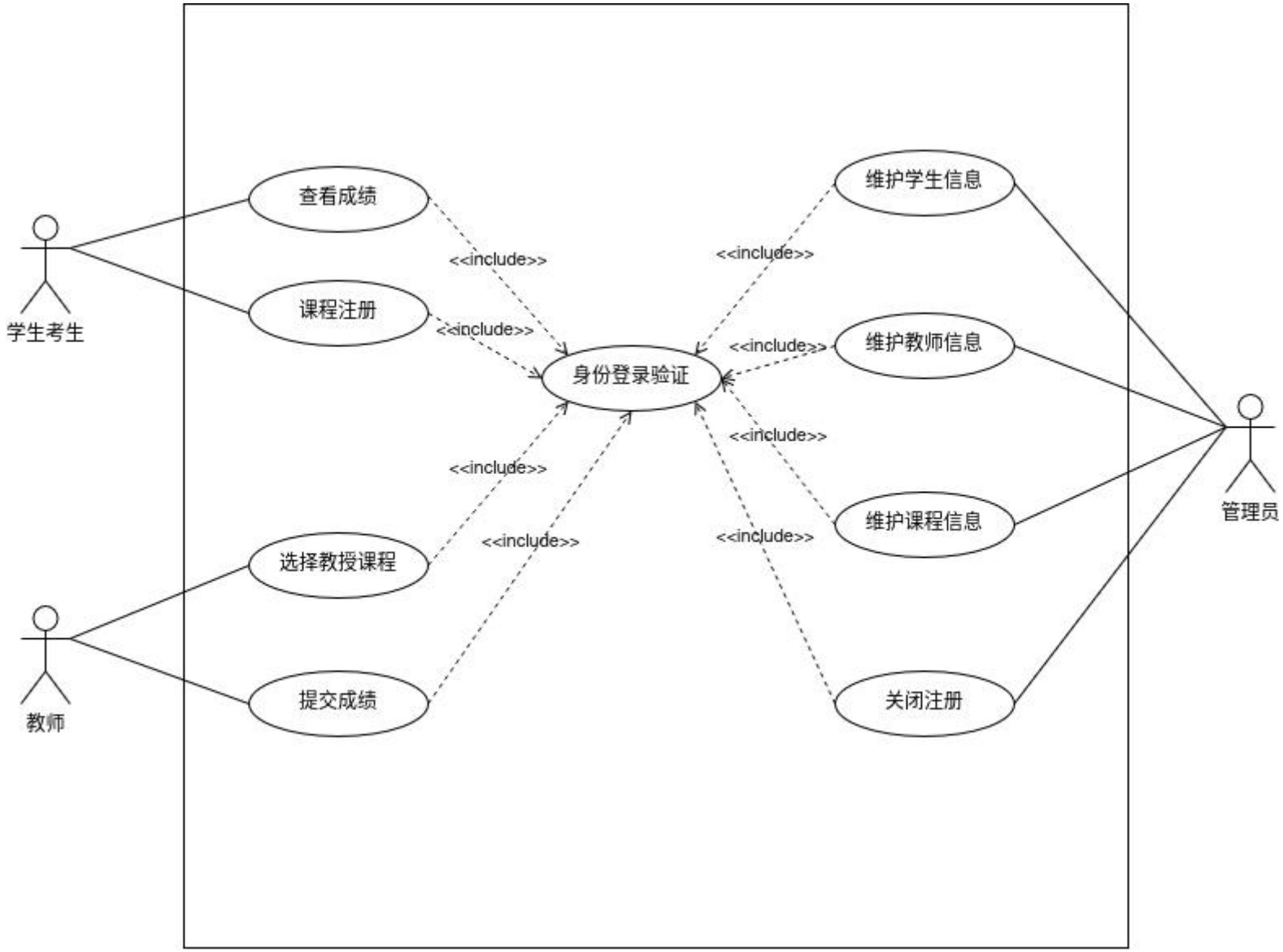
Model Documentation

Contextual notes, underlying assumptions, and enforced business rule specifications.

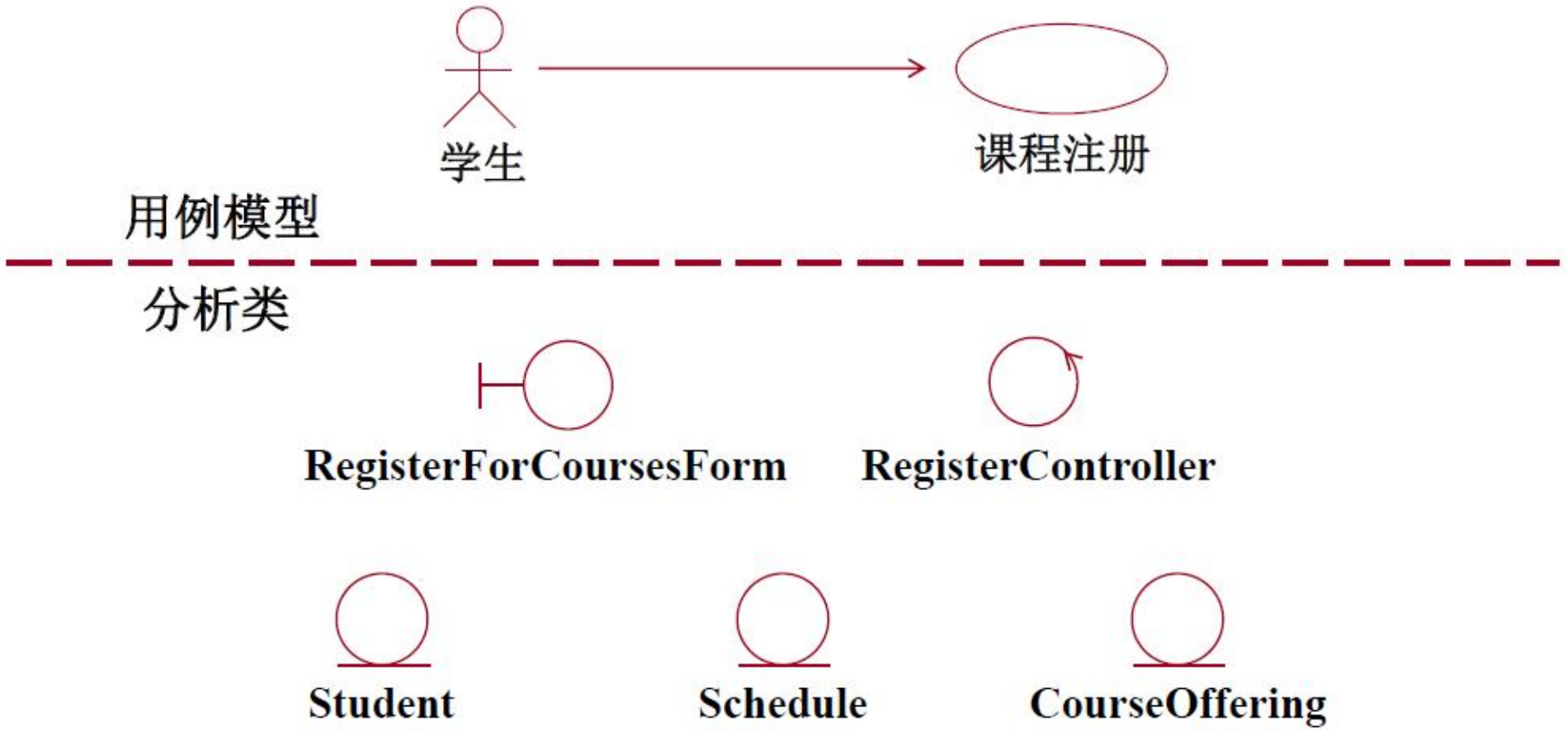
Foundation for Design

This package is the critical input for the Object-Oriented Design (OOD) phase, ensuring alignment with business goals.

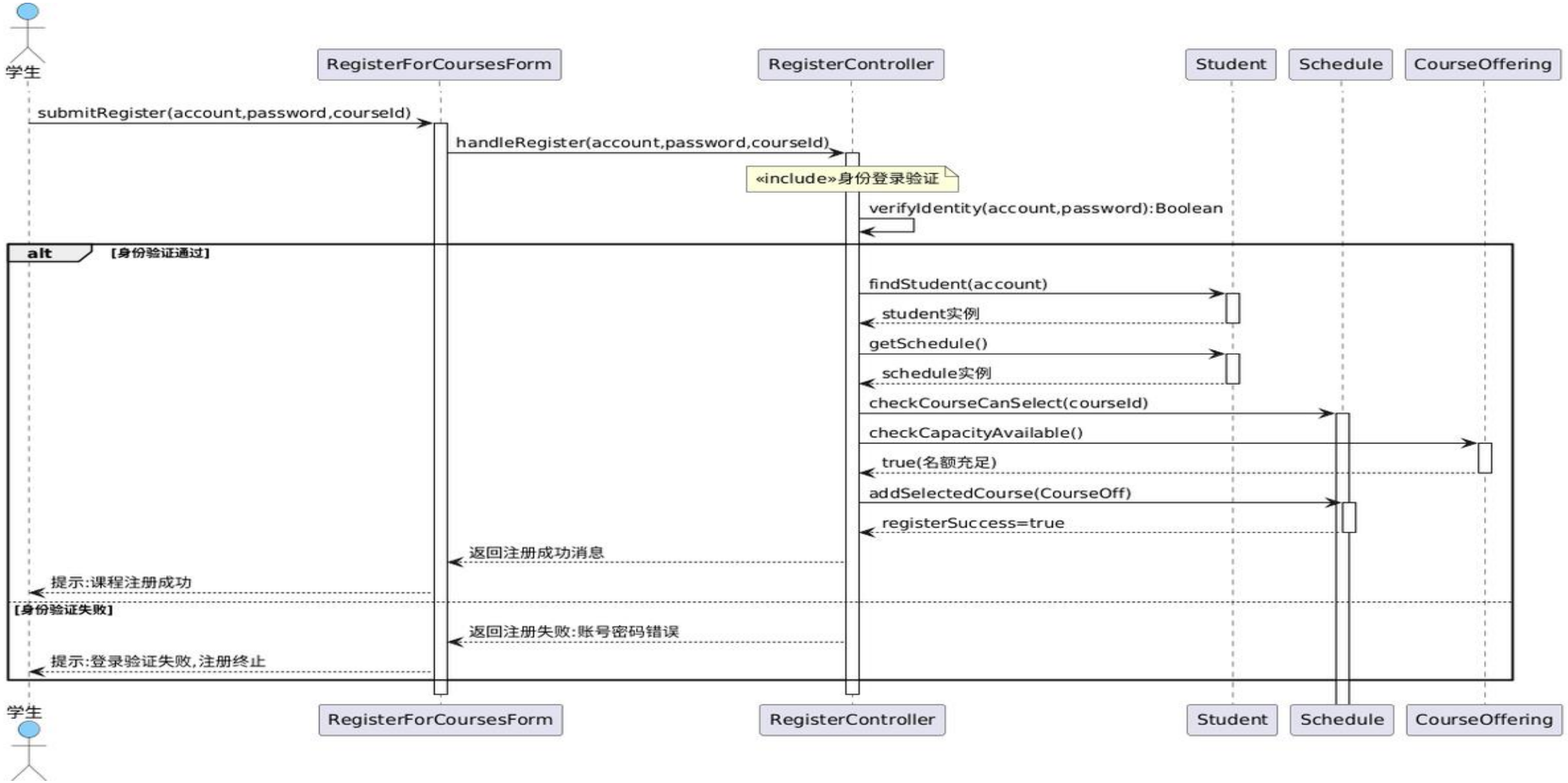
Example


查看成绩
课程注册
选择教授课程
提交成绩
学生考生
教师
身份登录验证
维护学生信息
维护教师信息
维护课程信息
关闭注册
管理员
<<include>>
<<include>>
<<include>>
<<include>>
<<include>>
<<include>>
<<include>>
<<include>>

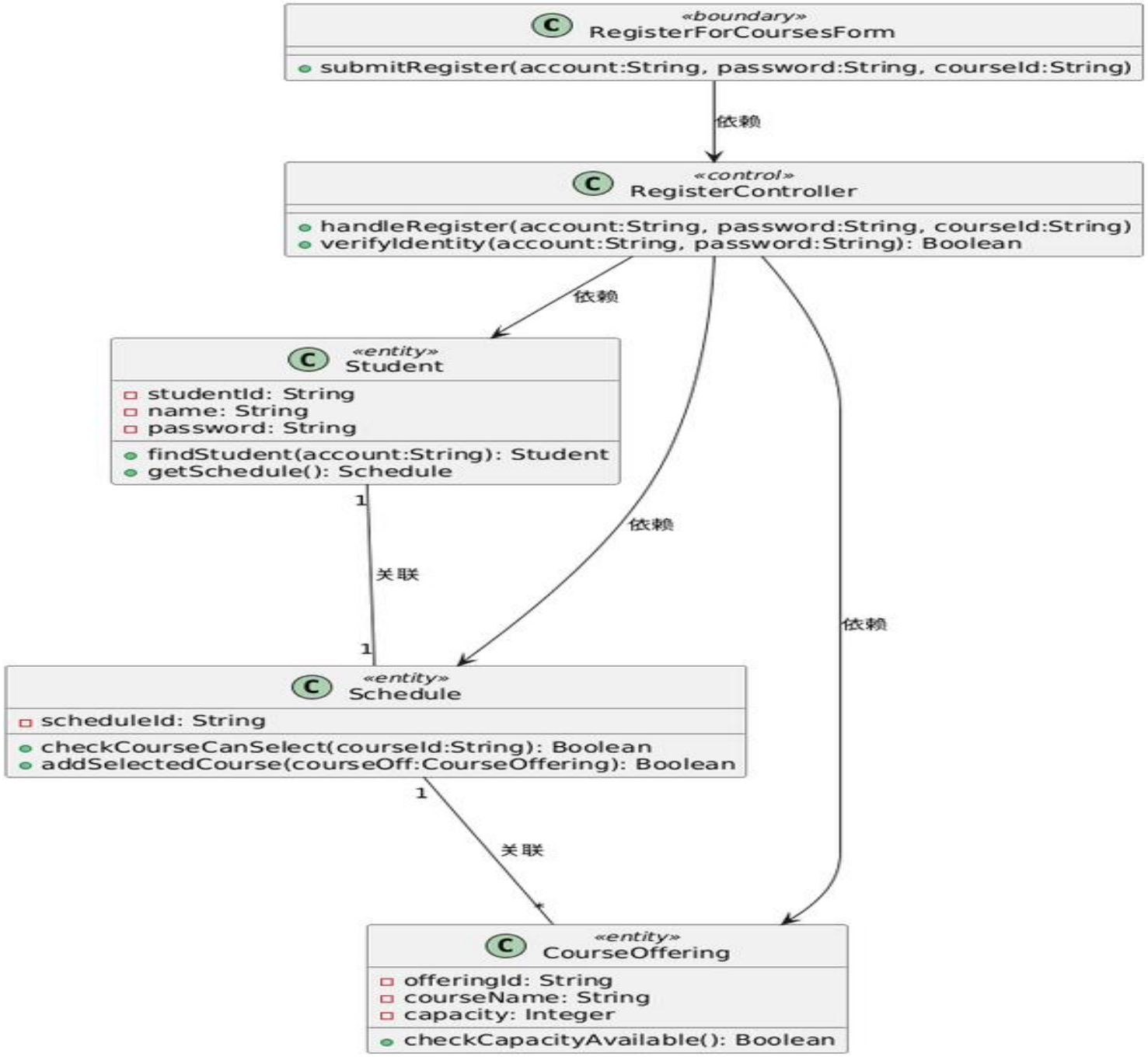
Example

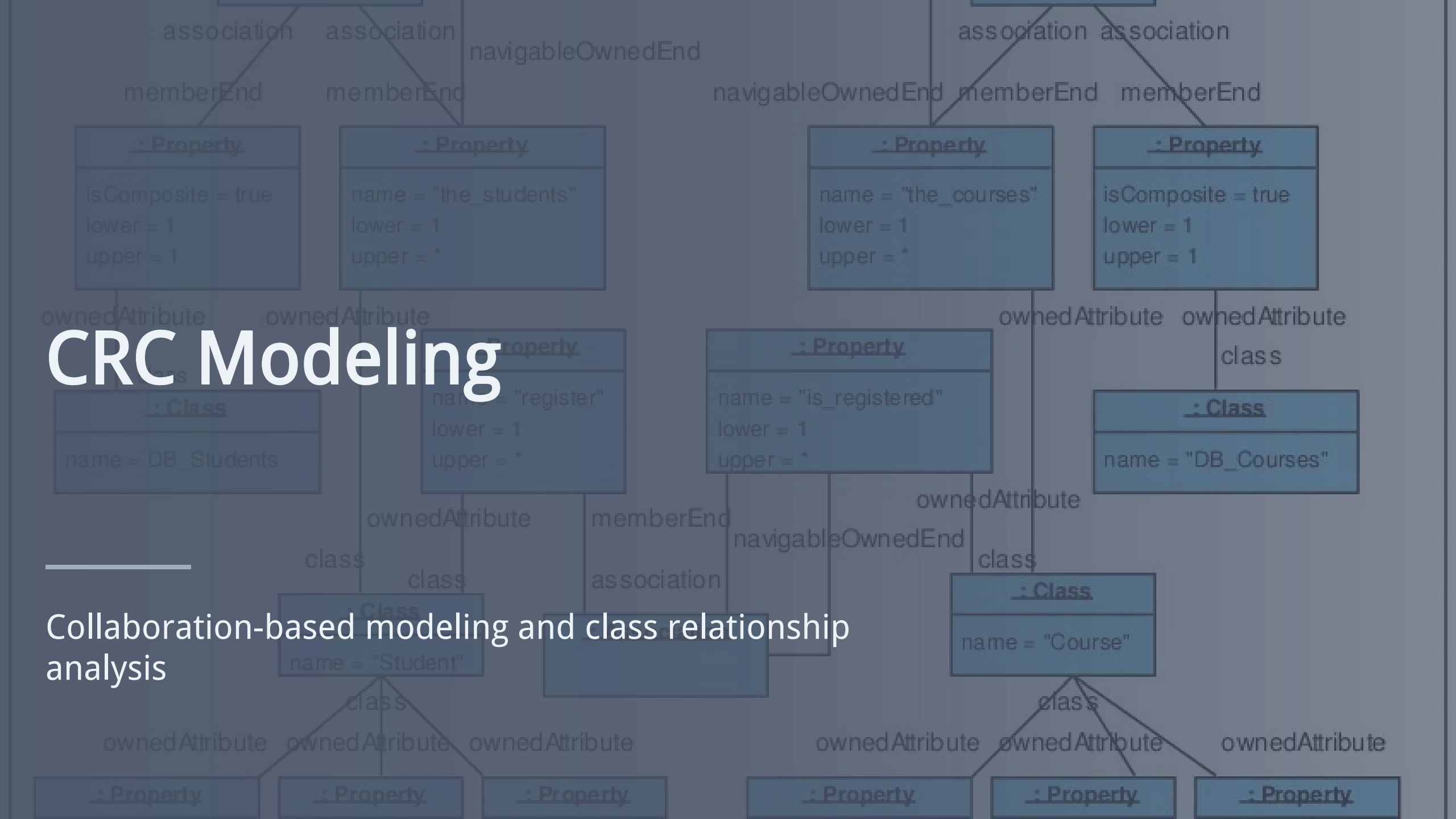

学生
课程注册
用例模型
分析类
RegisterForCoursesForm
RegisterController
Student
Schedule
CourseOffering

Example


学生
RegisterForCoursesForm
RegisterController
Student
Schedule
CourseOffering
submitRegister(account,password,courseId)
handleRegister(account,password,courseId)
«include»身份登录验证
verifyIdentity(account,password):Boolean
alt
【身份验证通过】
findStudent(account)
student实例
getSchedule()
schedule实例
checkCourseCanSelect(courseId)
checkCapacityAvailable()
true(名额充足)
addSelectedCourse(CourseOff)
registerSuccess=true
返回注册成功消息
提示:课程注册成功
【身份验证失败】
返回注册失败:账号密码错误
提示:登录验证失败,注册终止
学生
RegisterForCoursesForm
RegisterController
Student
Schedule
CourseOffering

Example


«boundary»
RegisterForCoursesForm
submitRegister(account:String, password:String, courseId:String)
依赖
«control»
RegisterController
handleRegister(account:String, password:String, courseId:String)
verifyIdentity(account:String, password:String): Boolean
依赖
依赖
依赖
«entity»
Student
studentId: String
name: String
password: String
findStudent(account:String): Student
getSchedule(): Schedule
1
关联
1
«entity»
Schedule
scheduleId: String
checkCourseCanSelect(courseId:String): Boolean
addSelectedCourse(courseOff:CourseOffering): Boolean
1
关联
*
«entity»
CourseOffering
offeringId: String
courseName: String
capacity: Integer
checkCapacityAvailable(): Boolean



CRC Modeling

Collaboration-based modeling and class relationship analysis

CRC Modeling Fundamentals

Collaboration-Based Design

CRC (Class-Responsibility-Collaboration) modeling helps identify how classes **work together** to fulfill system requirements.

Class

Represents a collection of similar objects.

Role:

Template for objects with common structure and behavior

Responsibilities

Attributes and operations the class encapsulates.

Types:

- Knowing (maintaining data)
- Doing (performing actions)

Collaborations

Classes required to provide information needed to complete responsibilities.

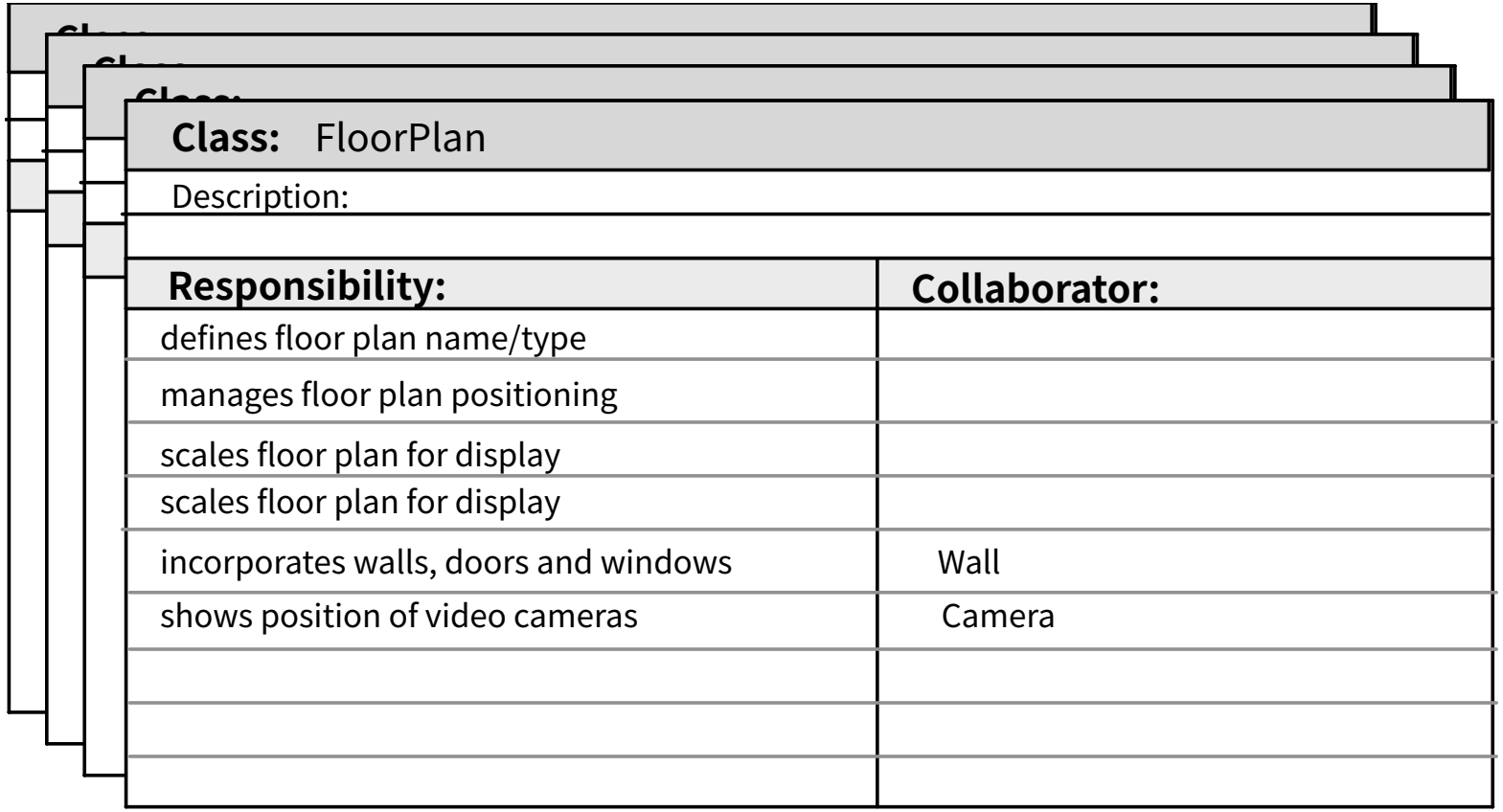
Purpose:

Identify relationships and dependencies between classes

CRC Model Example

CRC Index Card Format

CRC cards provide a **simple, tangible way** to represent classes and their responsibilities.



Class: FloorPlan

Description:

Responsibility:	Collaborator:
defines floor plan name/type	
manages floor plan positioning	
scales floor plan for display	
scales floor plan for display	
incorporates walls, doors and windows	Wall
shows position of video cameras	Camera

CRC Modeling

Responsibilities: Distribution Principles

Intelligent Distribution

System intelligence should be **distributed across classes** to best address the needs of the problem.

1 Distribute Intelligence

System intelligence should be distributed across classes to **best address problem needs**.

- Avoid "god classes" that do everything

2 State Generally

Each responsibility should be stated as **generally as possible**.

- General statements promote reuse

3 Co-locate Information

Information and related behavior should **reside within the same class**.

- High cohesion within classes

4 Localize Information

Information should be localized with **a single class**.

- Avoid distributing across classes

5 Share Responsibilities

Responsibilities should be **shared among related classes** when appropriate.

- Distributed responsibilities

Key Principle

These principles ensure **high cohesion, low coupling, and maintainable** object-oriented designs.

CRC Modeling

Reviewing the CRC Model

Validation Process

The CRC review process validates that classes and their responsibilities **can accommodate all use-case scenarios.**

CRC Review Steps

1. **Distribute Cards**
   All participants receive a subset of CRC cards. Cards that collaborate should be separated.
2. **Organize Use-Cases**
   All use-case scenarios should be organized into categories for systematic review.
3. **Read Use-Case**
   The review leader reads the use-case deliberately, identifying named objects.

Interactive Validation

4. **Pass Token**
   When a named object is encountered, pass a token to the person holding the corresponding class card.
5. **Describe Responsibilities**
   The card holder describes the responsibilities noted on their card.
6. **Validate & Modify**
   The group determines if responsibilities satisfy the use-case. If not, modifications are made to the cards.

Benefits of CRC Review

Validates
Ensures classes can fulfill requirements

Identifies
Finds missing classes or responsibilities

Improves
Enhances collaboration understanding

CRC Modeling

Analysis Packages

Organizing Model Elements

Analysis packages categorize model elements in a manner that **packages them as logical groupings**.

Package Purpose

Group related elements to improve **organization and manageability** of large models.

Grouping Strategy

By functionality, subsystem, or layer

Encapsulation

Hide internal details, expose public interfaces

Visibility Indicators

+ Public Visibility
Accessible from other packages

- Private Visibility
Hidden from all other packages

Protected Visibility
Accessible only to contained packages

Benefits of Packages

Organization
Logical grouping of related elements

Modularity
Clear boundaries and interfaces

Maintainability
Easier to understand and modify

Scalability
Supports large system development

CRC Modeling

Analysis Package Example

E-Commerce System Packages

Organizing an e-commerce system into **logical packages** with appropriate visibility.

User Management

- \+ Customer
- \+ UserAccount
- \+ Authentication

Product Catalog

- \+ Product
- \+ Category
- \# Inventory

Order Processing

- \+ Order
- \+ ShoppingCart
- \- PaymentGateway



Software Specification

Guidelines for writing clear and effective software specifications

The Challenge of Specification

The Specification Paradox

"Everyone knew exactly what had to be done until someone wrote it down!"

This quote highlights the fundamental challenge of software specification: **transforming tacit understanding into explicit, unambiguous documentation.**

Ambiguity

Natural language is often vague and open to multiple interpretations

Communication

Bridging understanding between diverse stakeholders

Complexity

Capturing all requirements without overwhelming detail

Change

Requirements evolve throughout the project lifecycle

Software Specification

Specification Guidelines

Best Practices

Put yourself in the reader's position - **"Would I understand this if I wasn't intimately familiar with the system?"**

1 Reader's Perspective

Put yourself in the reader's position. **"Would I understand this?"**

Write for your audience

2 Simple Style

Write in a **simple, unambiguous style** . Avoid jargon.

✓ Clarity over complexity

3 Consistent Notation

Use **consistent graphical notation** and terms.

Stay consistent

4 Substitute Definitions

When a term is defined, **substitute the definition** for other occurrences.

Replace with meaning

5 Visualize Structures

When a structure is described in words, **draw a picture** .

A picture is worth 1000 words

6 Express Equations

When symbolic equations are used, **express their meaning in words** .

Explain formulas

7 Work Examples

When a calculation is specified, **work at least two examples** .

Show calculations

8 Question Certainty

Search behind certainty statements - ensure **restrictions are realistic** .

Challenge assumptions

Exercise - Set 1

1. Which of these is not an element of an object-oriented analysis model? Answer: c
   a. Behavioral elements
   b. Class-based elements
   c. Data elements
   d. Scenario-based elements
2. Which of the following is not an objective for building an analysis model? Answer: c
   a. define set of software requirements that can be validated
   b. describe customer requirements
   c. develop an abbreviated solution for the problem
   d. establish basis for software design
3. The data flow diagram Answer: e
   a. depicts relationships between data objects
   b. depicts functions that transform the data flow
   c. indicates how data are transformed by the system
   d. indicates system reactions to external events
   e. both b and c
4. Which of the following items does not appear on a CRC card? Answer: c
   a. class collaborators
   b. class name
   c. class reliability
   d. class responsibilities

Exercise - Set 2

5. For purposes of behavior modeling a state is any
   a. consumer or producer of data.
   b. data object hierarchy.
   c. observable mode of behavior.
   d. well defined process.
   Answer: c
6. Attributes cannot be defined for a class until design has been completed.
   a. True
   b. False
   Answer: b
7. Operations are object procedures that are invoked when an object receives a message.
   a. True
   b. False
   Answer: a
8. UML activity diagrams are useful in representing which analysis model elements?
   a. Behavioral elements
   b. Class-based elements
   c. Flow-based elements
   d. Scenario-based elements
   Answer: d